컴퓨터구조

Presented by
오 세 춘

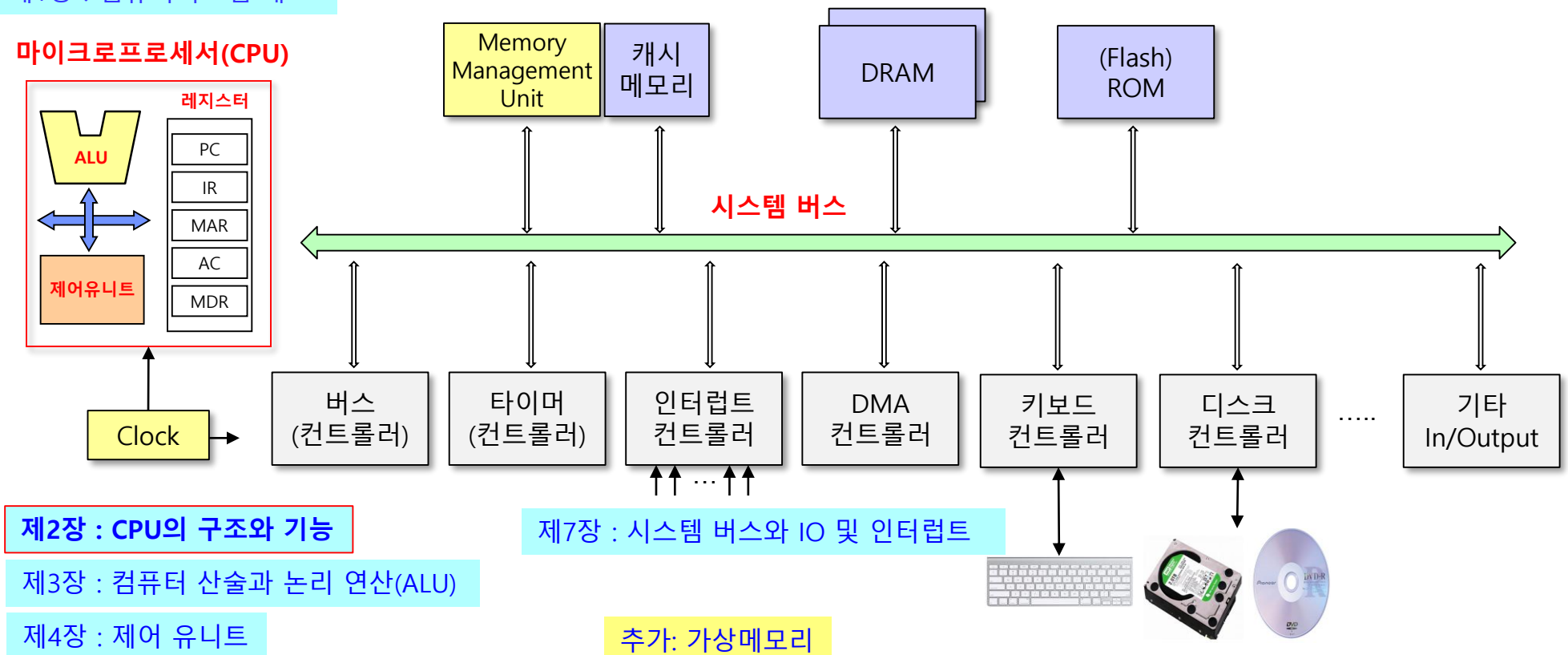
2-1장 CPU의 구조와 기능

교재 주요 구성

■ 컴퓨터 시스템의 구성 – 교재 및 강의 구성 내용

제1장 : 컴퓨터시스템 개요

제5장 : 기억장치



제2장 : CPU의 구조와 기능

제3장 : 컴퓨터 산술과 논리 연산(ALU)

제4장 : 제어 유니트

제7장 : 시스템 버스와 IO 및 인터럽트

추가: 가상메모리

제6장 : 보조기억장치

제8장 : 고성능 컴퓨터시스템 구조

배울 내용

■ 목차

✓ 2.1 CPU의 기본 구조
✓ 2.2 명령어 실행
✓ 2.3 명령어 파이프라이닝
✓ 2.4 명령어 세트

약 1주간 학습

약 1주간 학습

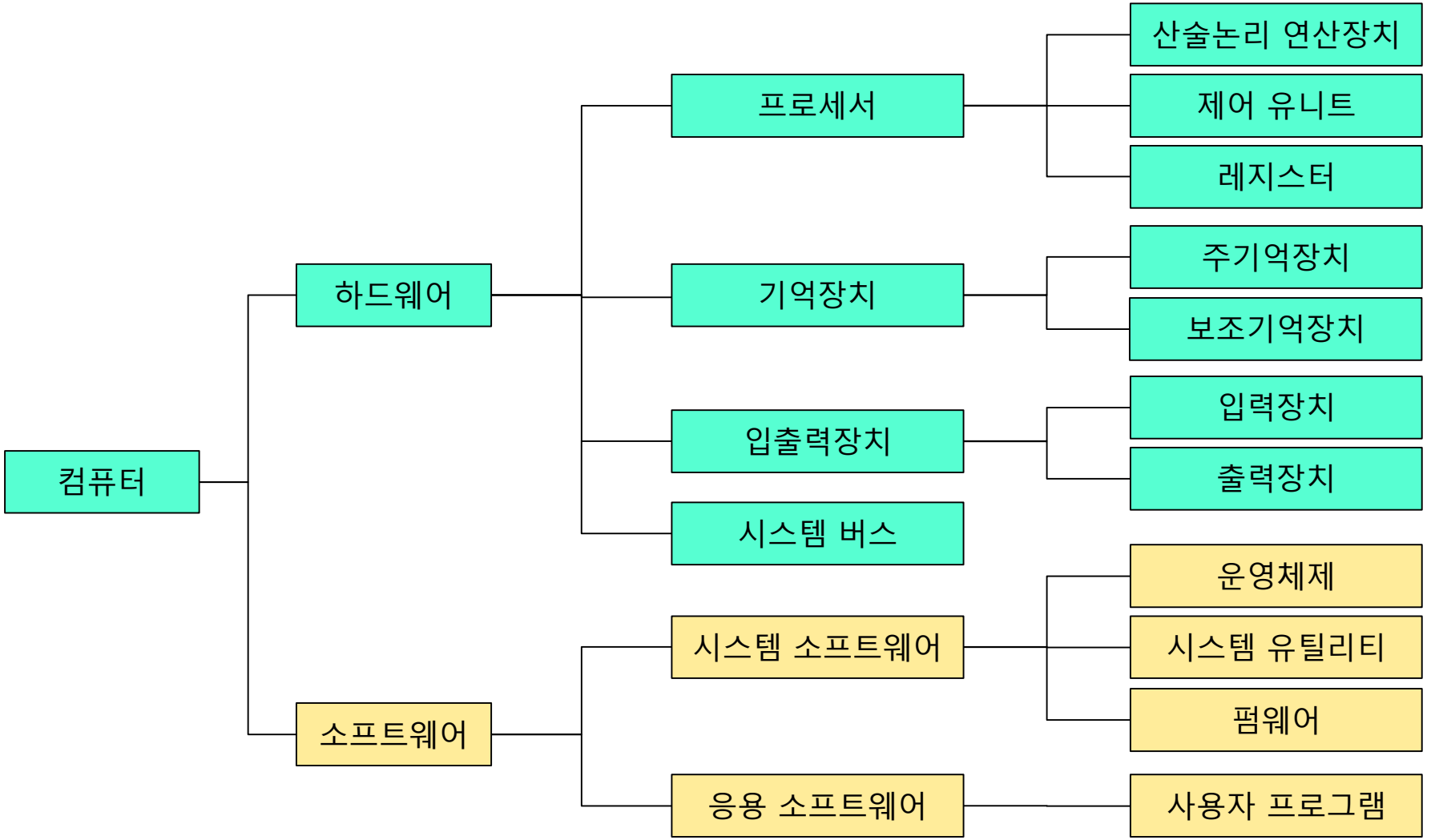
약 1주간 학습

구분	클록(Clock)	클럭(Clock)
표준	O (표준어)	X (비표준어)
근거	국립국어원 외래어 표기법	관용적 굳어짐 (오랜 기간 사용)
빈도	상대적으로 낮음	일상, IT 업계에서 압도적으로 높음
사용처	교과서, 시험, 공문서 등	IT 전문 매체, 기술 블로그, 실생활 대화

두 표현 모두가 사용이 가능한 용어

2 CPU의 구조와 기능

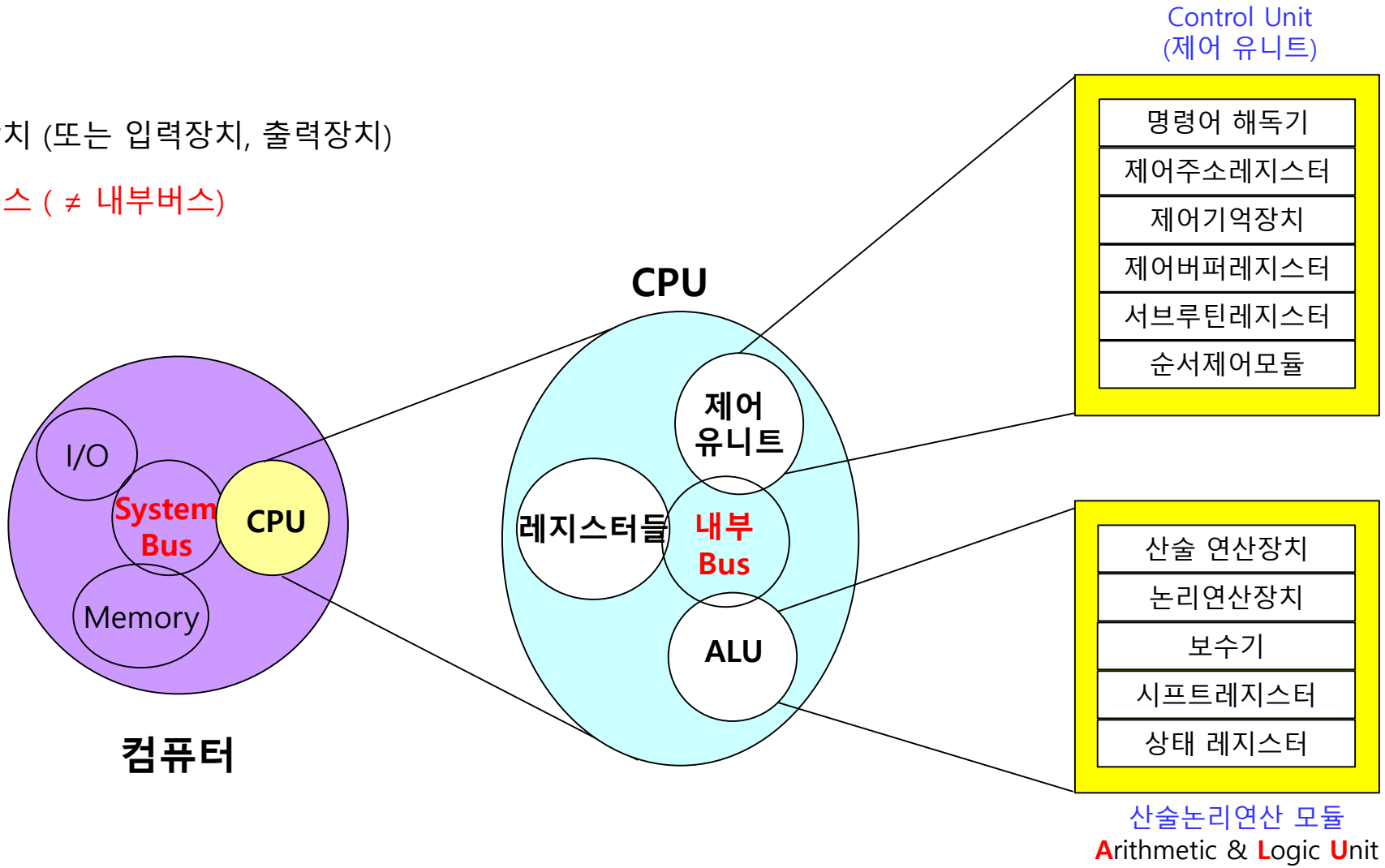
■ 컴퓨터의 기본적인 구성요소



2.1 CPU의 기본 구조

■ 컴퓨터의 기본적인 구성요소

1) CPU
2) 기억장치
3) 입출력장치 (또는 입력장치, 출력장치)
4) 시스템버스 (≠ 내부버스)



2.1 CPU의 기본 구조

■ 주요 기능

✓ 컴퓨터 시스템 전체의 데이터 흐름을 제어

✓ 주기억장치로부터 프로그램을 읽어 명령을 해독, 실행하고 다음에 실행해야 할 명령어를 결정

✓ CPU의 세부 동작 순서의 예 (무한 반복)

① **명령어 인출**(Instruction Fetch)

❖ 주기억장치로부터 명령어 하나를 CPU 내부로 읽어온다

② **명령어 해독**(Instruction Decode)

❖ 수행해야 할 동작을 결정하기 위하여 읽어온 명령어를 해독(분석)한다

③ 데이터 인출(Data Fetch)

❖ 명령어 실행을 위하여 데이터가 필요한 경우에는 기억장치 혹은 I/O 장치로부터 그 데이터를 추가적으로 읽어온다

④ **데이터 처리**(Data Process)

❖ 데이터에 대한 이동, 산술적 혹은 논리적 연산을 수행

⑤ 데이터 저장(Data Store)

❖ 수행한 결과를 저장

- ①, ②단계는 모든 명령어가 반드시 공통적으로 수행되며
  나머지 단계는 명령어의 종류에 따라 생략되는 경우도 있다.
- CPU의 성능을 높이기 위해 각 세부 단계를 **파이프라인** 기술에 적용
- **CPU에 따라 명령어 인출, 명령어 실행의 두 파트로 설명하는 경우도 있음**

2.1 CPU의 기본 구조

P63~p64

■ 프로세서(CPU)의 구성 요소

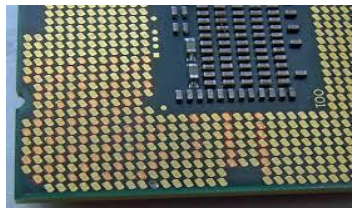
① ALU (Arithmetic & Logical Unit) → 산술, 논리 연산을 전담

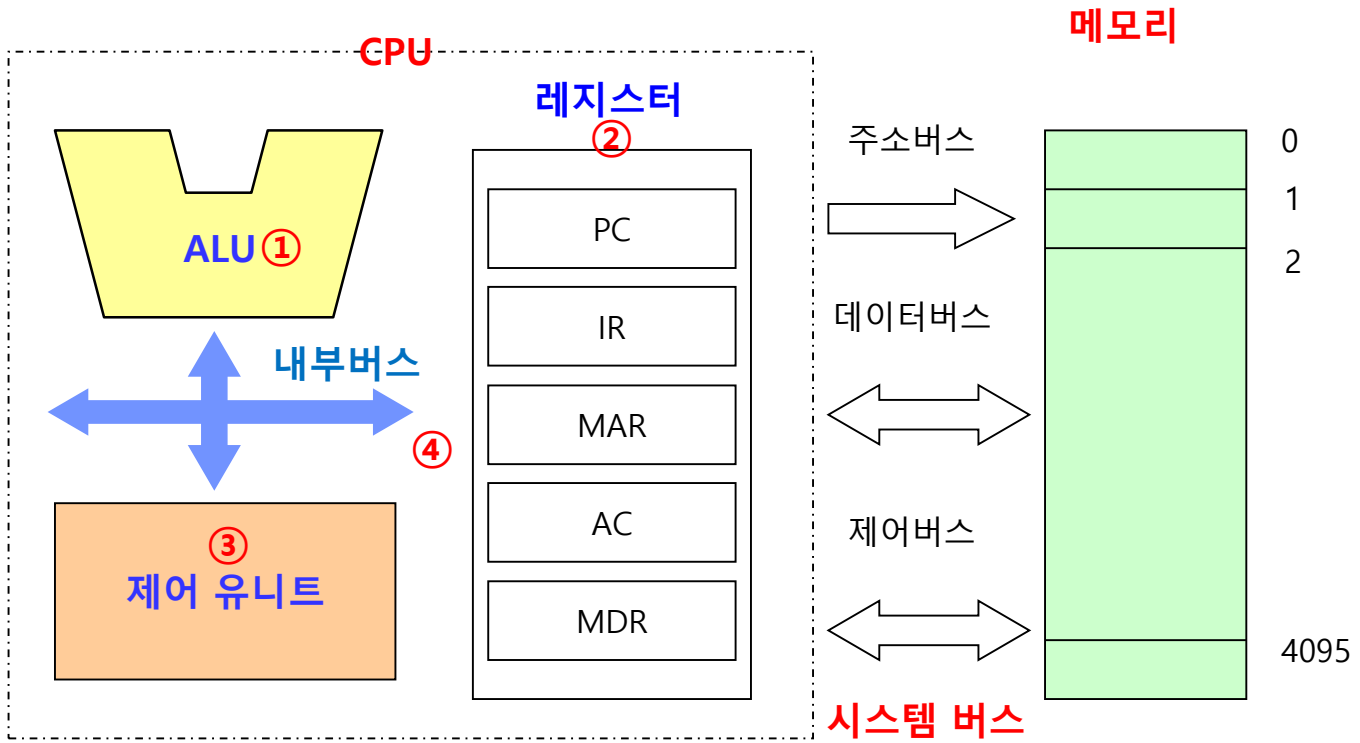
② 레지스터 (Register) → CPU 내부의 빠른 기억장치, 범용 및 CPU를 구동하기 위한 특수한 용도로 사용

③ 제어 유니트 (Control Unit) → 명령어를 실행하면서 모든 논리회로를 차례에 맞게 구동

④ CPU 내부 버스 (Internal Bus) → CPU 내부 연결용 버스 (외부 연결용도인 시스템 버스와는 다름)





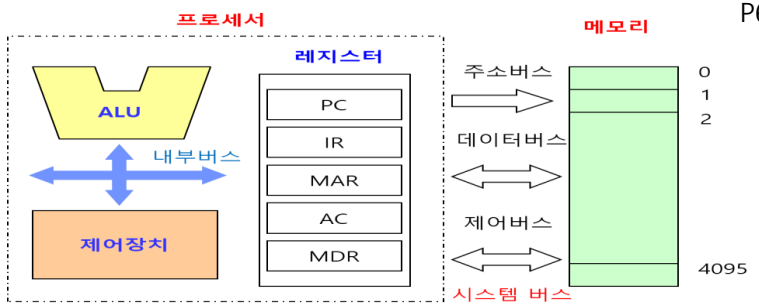


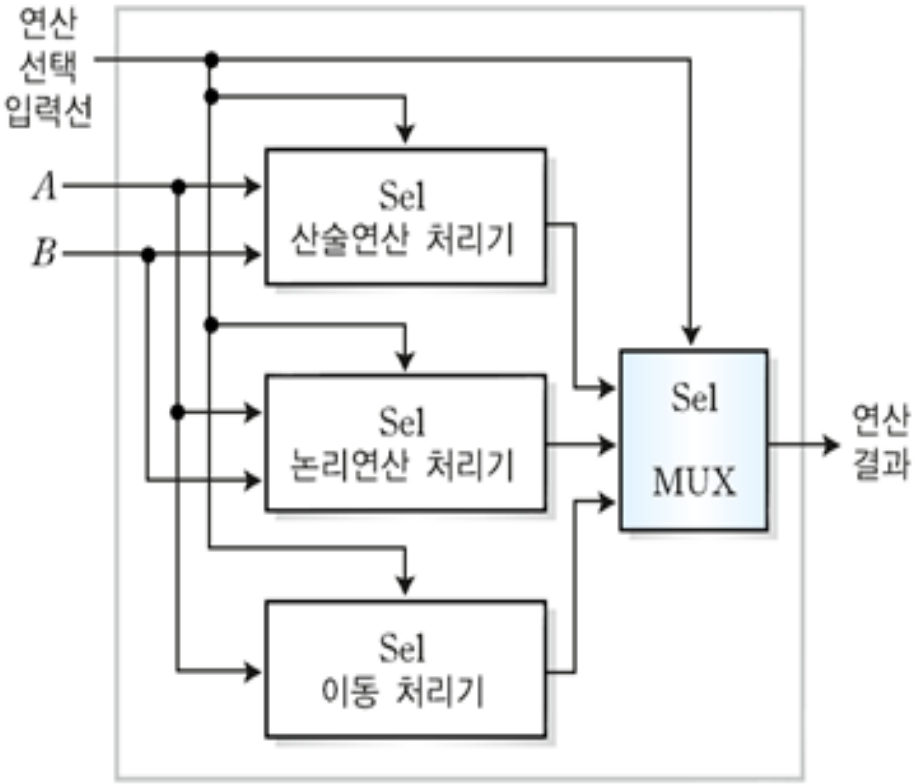
2.1 CPU의 기본 구조

P63~p64

1) ALU (Arithmetic & Logic Unit)

- ✓ 산술논리 연산장치
- ✓ 제어 유니트(제어 장치)의 지시를 받아 각종 **산술 연산**과 **논리 연산**을 전문적으로 수행하는 회로들로 이루어진 하드웨어 모듈
  - ➢ 산술(Arithmetic) 연산 : **+, -, ×, ÷**
  - ➢ 논리(Logic) 연산 : AND, OR, NOT, XOR 등
  - ➢ 산술/논리 Shift, Rotate 등
  - ➢ 연산 내용은 3장에서 상세 학습
- ✓ 연산의 결과를 지정된 레지스터에 저장
- ✓ 연산 결과의 상태를 상태 레지스터로 전달
  - ➢ Carry, Negative, Zero, Overflow .....
- ✓ 일부 프로세서는 하나의 프로세서 내에 여러 개의 ALU를 사용하기도 하고 또한 별도의 고정소수점 연산 전용 ALU, 부동소수점 연산 전용 ALU를 갖기도 한다.





2.1 CPU의 기본 구조

P63~p64

2) 레지스터 (Register)

CPU 마다 조금씩 다르나 대부분 유사한 기능의 레지스터들을 갖는다

- ✓ CPU 내부에 존재하며 명령어/데이터를 일시적으로 저장하는 **가장 빠른 메모리**
- ✓ 용도별 분류
  - ➢ 일반 용도 레지스터 (General Purpose register)
    - ❖ 프로그래머(컴파일러)가 임의로 사용하는 레지스터들 → 빠른 메모리 기능
    - ❖ RISC 프로세서에서는 일반 용도 레지스터의 수를 늘려 처리 속도를 향상
  - ➢ 특수 용도 레지스터 (Special Purpose register)
    - ❖ CPU의 동작을 위해 특수한 용도로만 사용되는 레지스터
    - ❖ 사용자(CPU 외부)에게 보이지 않는 레지스터도 있다

1) 명령어/데이터 처리 레지스터

☞ PC, MAR, MDR, IR, AC

2) 주소 지정 레지스터

☞ Base Register, Index Register, Segment Pointers, Stack Pointer

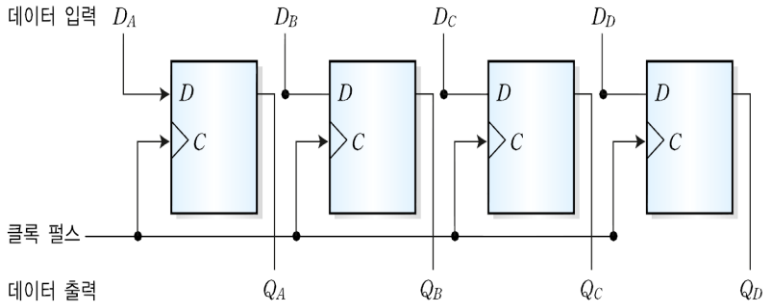
3) 상태 레지스터 (Status Register)

☞ Flag Register(=PSWR : Program Status Word Register)



< 메모리 계층 구조 >

클록	D	$Q(t+1)$
1	0	0
1	1	1



< 4비트 PIPO 레지스터 >

2.1 CPU의 기본 구조

2-1) 명령어/데이터 처리 레지스터

① PC (Program Counter)

- 다음에 실행될 명령어의 주기억장치 주소를 저장

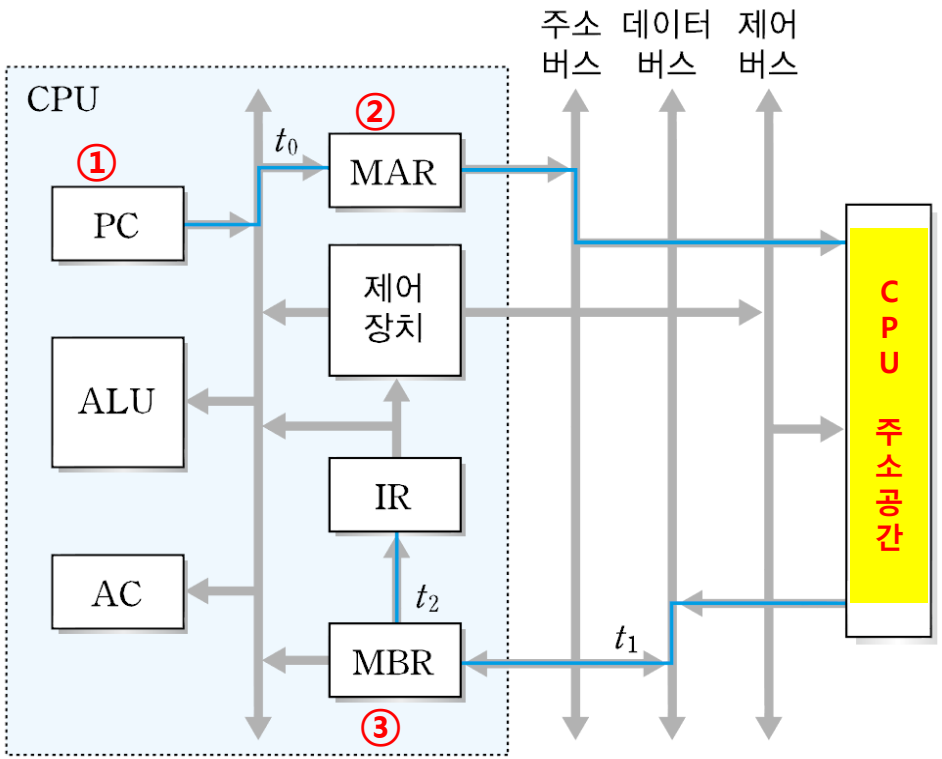
② MAR (Memory Address Register)

- 접근하고자 하는 장치의 주소 값을 저장
- MAR과 CPU 주소 공간(기억장치 공간)과 관계
  - CPU 주소 공간이 8192 워드 → 2^{13} → 13bit MAR 필요
  - CPU 주소 공간이 4G 워드 → 2^{32} → 32bit MAR 필요
- 우리 교재에서는 하나의 주소에 한 바이트가 아닌 워드를 지정

③ MBR (Memory Buffer Register)

- MDR (Memory Data Register) 라고도 함
- 선택된 장치에서(로) 읽은 데이터, 또는 쓸 데이터를 저장
- MBR과 워드(Word)와의 관계
  - 위드 길이가 16비트이면 일반적으로 MBR도 16비트로 구성

CPU는 System Bus와는 **MAR**, **MBR 레지스터**를 통해서만 연결된다. 즉 MAR과 MBR 레지스터의 각 비트들은 CPU 외부의 신호선 핀(pin)들과 연결되어져 있음.



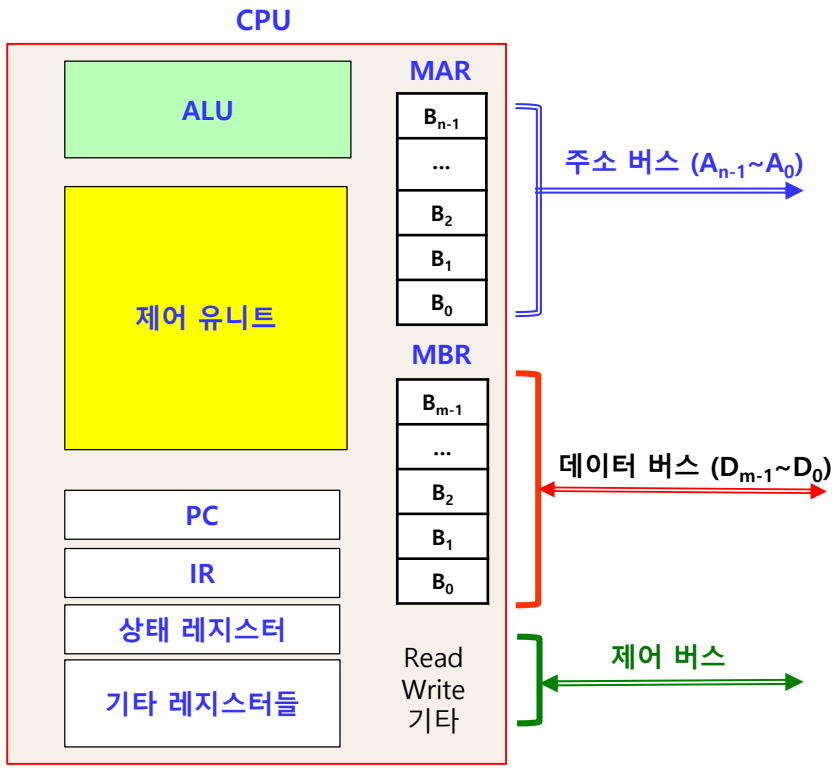
인텔 8088 CPU의 경우는 8086과 동일한 16비트 워드 구조이나 핀 수를 줄이기 위해 데이터 버스(MBR)는 8비트로 하여 2번을 불러온다.

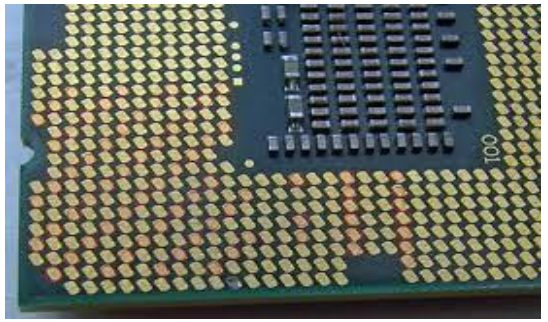
2.1 CPU의 기본 구조

2-1) 명령어/데이터 처리 레지스터 - 계속

✓ 참고: MAR, MBR 레지스터와 CPU 핀

➢ CPU 내부의 MAR, MBR 레지스터의 각 비트들 및 개별 제어 신호선들이 CPU 외부의 핀 하나 하나와 개별적으로 연결됨







2.1 CPU의 기본 구조

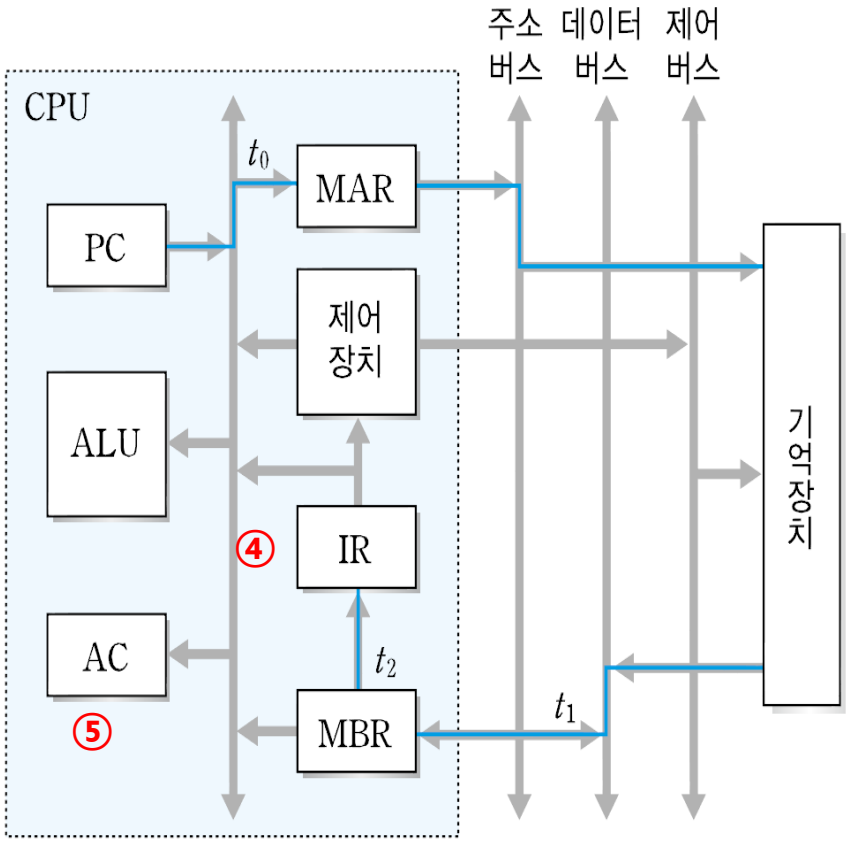
2-1) 명령어/데이터 처리 레지스터 - 계속

④ IR (Instruction Register)

- 현재 수행하고 있는 명령어를 저장
- 이 레지스터 값은 연산에 사용되지 않고 명령어 수행을 제어하기 위해 사용 (제어 유니트가 사용)

⑤ AC (ACcumulator = ACC)

- 누산기
- 연산기능 수행 시 중심이 되는 ALU와 연결
- 연산에 사용되는 데이터 및 연산결과를 임시적으로 기억



2.1 CPU의 기본 구조

2-2) 주소 지정 레지스터

2.4.3 주소지정 방식에서 상세 학습

✓ 주기억장치(메모리) 사용을 편하게 해주는 기능의 주소관련 레지스터들

① Base Register

- ❖ 프로그램 루틴이나 데이터 블록에 대해 기준이 되는 주소를 기억

② Index Register

- ❖ 배열 내에서의 주소와 같은 반복 계수를 기억시키는 용도
- ❖ 일반적으로 Base Register와 함께 사용하며, 자동으로 증가/감소 기능이 있는 경우도 있음

③ Segment Pointers (주로 인텔 CPU에서 사용)

- ❖ 세그먼트 번호(베이스)와 세그먼트 내부의 변위(오프셋)로 구성되어야 한다.
- ❖ 세그먼트 레지스터는 세그먼트의 베이스 주소를 갖고 있음.

④ Stack Pointer(**SP**)

- ❖ 스택 포인터: 스택의 위치를 나타내는데 사용
- ❖ push 명령 → SP 증가, pop 명령 → SP 감소

2-2) 주소 지정 레지스터 - 스택 포인터 (Stack Pointer) 레지스터

- ✓ 많은 프로세서는 하나 이상의 스택 포인터 레지스터를 내장
- ✓ 스택(Stack)
  - ➢ CPU가 임시로 저장할 데이터를 **외부 메모리(스택 영역)에 데이터를 빠르게 저장/읽기 위한 메모리 블록**
    - ❖ 특별히 저장할 메모리 주소를 지정하지 않아도 자동으로 저장되는 메모리 위치를 관리(증가 또는 감소)
    - ❖ 스택 포인터(SP)는 스택 영역에 저장될 또는 읽힐 주소를 저장
    - ❖ 초기에 프로그램이 CPU의 SP 초기치를 설정하면 이후 CPU가 SP를 자동으로 관리
- ✓ 임시 저장이 필요한 레지스터의 내용을 스택 영역에 간편하게 기록하고 간편하게 읽어 온다
  - ① 데이터의 임시 저장할 경우 푸시(push) 명령어 사용 → SP가 가리키는 메모리 위치에 저장
  - ② CPU는 다른 명령을 실행
  - ③ 데이터 복구 시 팝(pop) 명령어 실행 → 바로 이전 push 명령에 의해 저장된 값이 복원됨
  - ➢ 한 개의 push, pop 명령 후에 CPU는 자동으로 SP를 조정
- ✓ 인터럽트, 서브루틴에서 주로 사용
- ✓ LIFO (Last-In First-Out) 구조
  - ➢ push, pop 명령의 순서가 다르면 엉뚱한 값이 오기 때문에 주의가 필요

2.1 CPU의 기본 구조

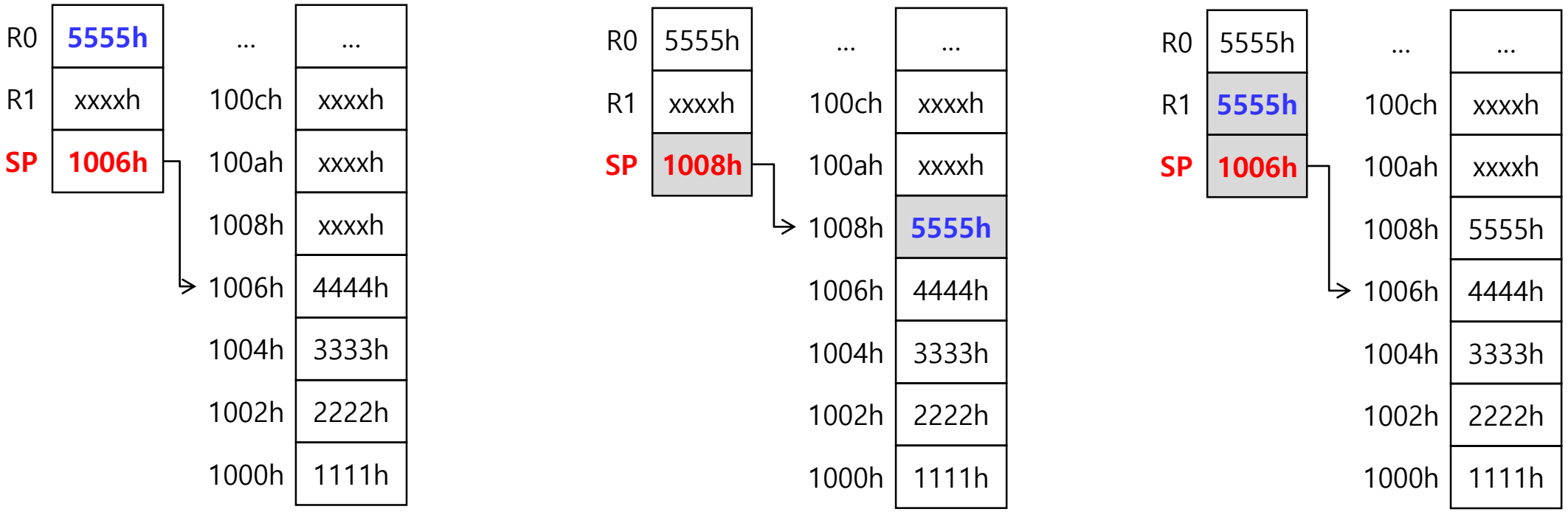
■ 참고 : 스택의 동작 원리

✓ 예: 기억장치는 바이트 단위로 구성, 데이터 16비트

push/pop 명령시의 SP의 관리방법(증가/감소 등)은 CPU 마다 다르다
→ 이 페이지는 ARM의 예제임

CPU가 'push R0' 명령 실행
① CPU는 SP를 자동으로 증가(1008h)
② SP가 가리키는 주소(1008h)에 R0 값 저장

CPU가 'pop R1' 명령 실행
① SP가 가리키는 주소(1008h)의 값을 R1으로
② CPU는 SP를 자동으로 감소(1006h)



(a) 초기 상태 (b) push R0 실행 후 (c) pop R1 실행 후

push 명령어를 여러 개 사용하는 경우?

2.1 CPU의 기본 구조

■ 참고 : 스택의 동작 원리 (8086 데모 – stack.asm)

향후 실습 예정임

```
sseg segment para stack 'stack'
    db 64h dup(?)
sseg ends
;-------------------------------------------------------
dseg segment
data1 db 'Stack Test','$'
dseg ends
;-------------------------------------------------------
cseg segment
main proc far
    assume ss:sseg, ds:dseg, cs:cseg

    mov ax, 1234h          ; ax 레지스터에 1234h 값 넣음
    mov bx, 5678h          ; bx 레지스터에 5678h 값 넣음

    push ax                ; ax 내용을 스택에 저장(백업)
    push bx                ; bx 내용을 스택에 저장(백업)

; 다른 작업으로 ax, bx가 변경되므로 ax, bx를 저장
    mov ax,0               ; ax를 다른 값(0)으로 변경
    mov bx,0               ; bx를 다른 값(0)으로 변경

; 다른 작업 이후에 원래의 ax, bx를 복원
    pop  bx                ; 스택에 저장된 bx(5678h)를 복원
    pop  ax                ; 스택에 저정된 ax(1234h)를 복원
```

```
; pop 명령은 마지막 push된 순서로 차례로 꺼내어 진다
    push ax                ; ax = 1234h
    push bx                ; bx = 5678h

    mov  ax,0
    mov  bx,0

    pop  ax                ; 나중에 저장된 5678h가 ax로 복원
    pop  bx                ; 그 이전에 저장된 1234h가 bx로 복원

    ret

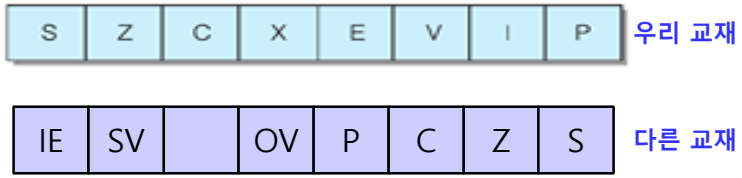
main endp
cseg ends
end main
```

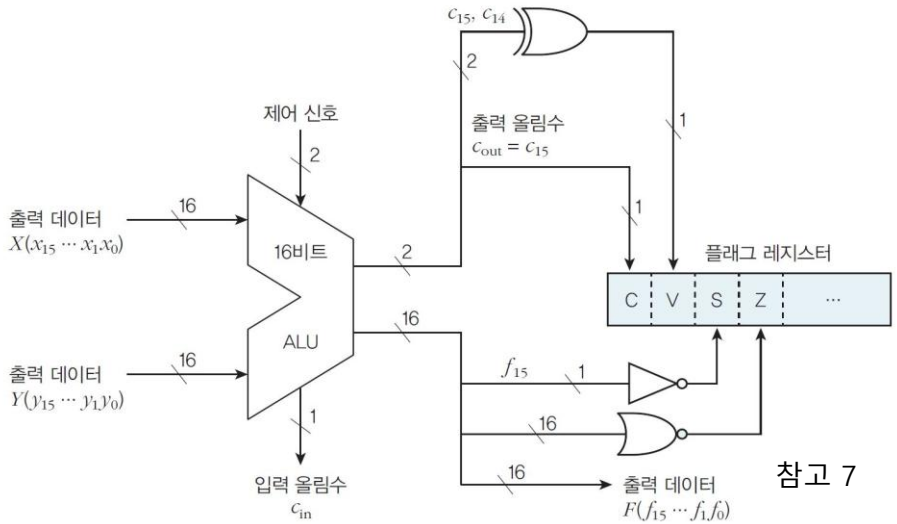
LIFO (Last-In First-Out)

8086은 앞 페이지의 ARM과 SP의 처리방법이 다르다
(SP의 증가/감소 방향 등등)
따라서 특별히 SP의 내용을 직접 접근하지 않을 경우라면
스택과 SP의 개념만 이해하면 된다

2-3) 상태 레지스터 (Status Register)

- ✓ Flag 레지스터 또는 PSWR(Program Status Word Register) 레지스터 라고도 부름
- ✓ 매 순간 ①CPU의 상태를 보여주는 기능 + ②CPU 기능의 제어
  - ➢ 현재 수행중인 명령어는 **실행의 결과(상태)를 상태 레지스터의 해당 플래그에 기록**
  - ➢ 조건부 명령어들은 명령 실행시에 상태 레지스터의 **해당 플래그를 참고하여 조건부 수행 실시**
- ✓ 플래그(flag) 들의 모임
  1) 명령어 실행의 결과(조건) 플래그 : 부호, 오버플로우, 캐리, 인터럽트 발생 여부, ...
  2) 제어 플래그 : Interrupt 마스크, 수퍼바이저 모드, ...





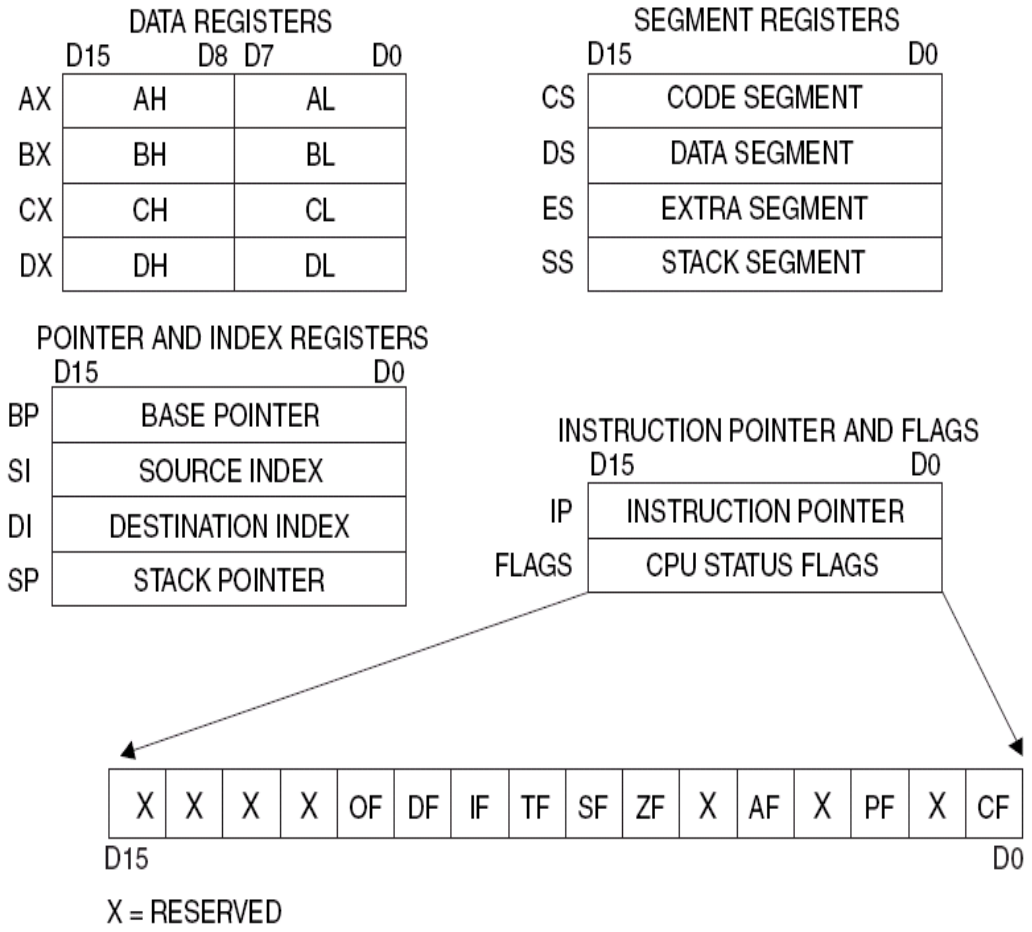
참고 7

조건 플래그
- 부호 (S, sign flag) → MSB 값
- 제로 (Z, zero flag) → 연산결과가 0
- 자리올림수 (C, carry flag) → MSB에서의 캐리 값
- 패리티 (P, parity flag) → 우리 교재에는 없음
- 오버플로우 (V=OV, overflow flag)
- 동등 (E, equal) → 비교 결과가 동일

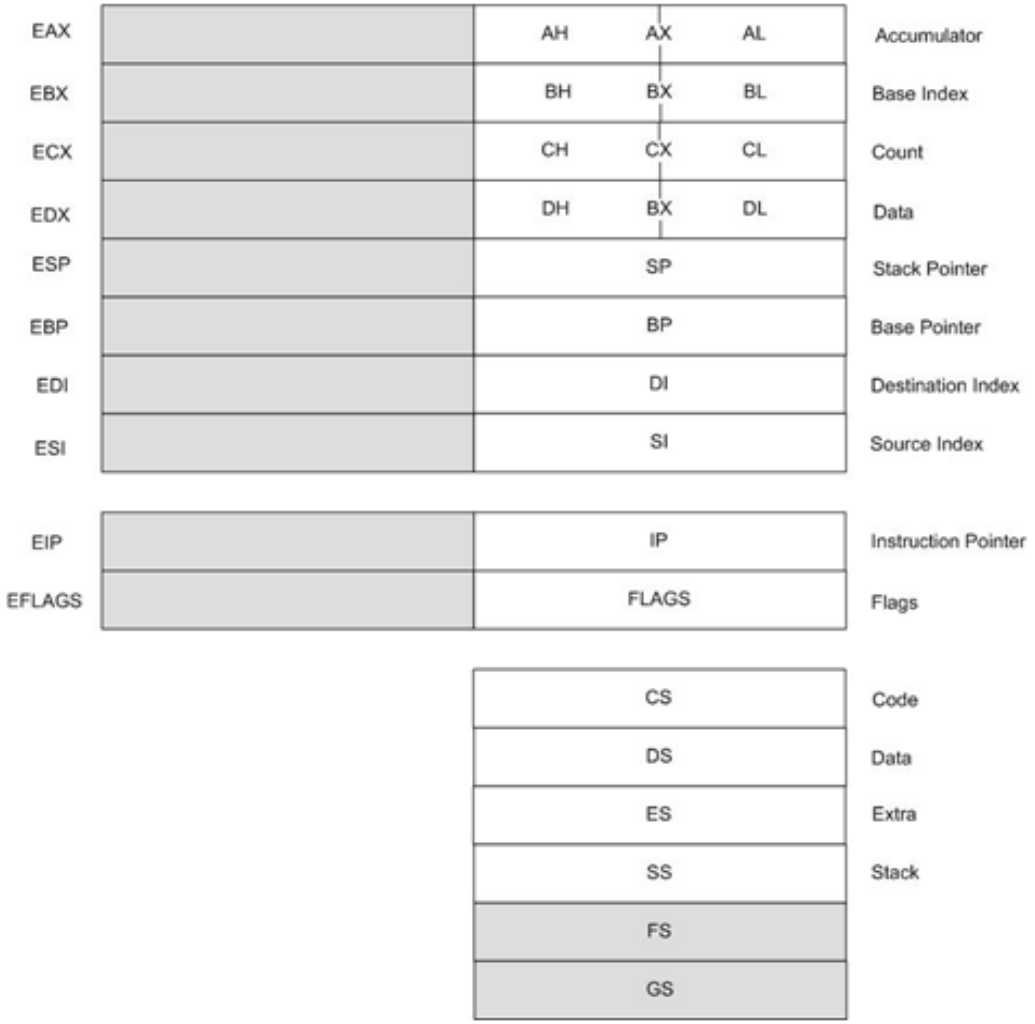
제어 플래그
- 인터럽트 (IE= I(우리교재), Interrupt Enable flag)
- 운영체제 (SV=P(우리교재), supervisor mode flag)

2.1 CPU의 기본 구조

■ 참고: 인텔 80x86 계열 프로세서의 레지스터



< 8086 레지스터 집합 : 14개(16b)



< 80486 레지스터 집합 : 10개(32b) + 6개(16b)>

2.1 CPU의 기본 구조

■ 참고: 인텔 80x86 계열 프로세서의 레지스터

향후 실습 예정임

- ✓ 주소 지정 레지스터
  - ➢ 프로그램의 실행 → CS:IP
  - ➢ 기존 데이터의 처리 → DS:SI
  - ➢ 추가 데이터의 처리 → ES:DI
  - ➢ 스택의 처리 → SS:SP
  - ➢ 추가적인 주소 지정 → BP
- ✓ 범용 레지스터
  - ➢ AX (AH, AL)
  - ➢ BX (BH, BL)
  - ➢ CX (CH, CL)
  - ➢ DX (DH, DL)
- ✓ Status Register (Flags)

< 8086 DEBUG 화면 >

```
C:\DOS>debug
-a
073F:0100 mov ax,2
073F:0103 dec ax
073F:0104 cmp ax,0
073F:0107 jne 103
073F:0109 nop
073F:010A nop
073F:010B
-r
AX=0000  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=073F  IP=0100   NV UP EI PL NZ NA PO NC
073F:0100 B80200        MOV     AX,0002
-p

AX=0002  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=073F  IP=0103   NV UP EI PL NZ NA PO NC
073F:0103 48            DEC     AX
-p

AX=0001  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=073F  IP=0104   NV UP EI PL NZ NA PO NC
073F:0104 3D0000        CMP     AX,0000
-
AX=0001  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=073F  IP=0107   NV UP EI PL NZ NA PO NC
073F:0107 75FA          JNZ     0103
-p

AX=0001  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=073F  IP=0103   NV UP EI PL NZ NA PO NC
073F:0103 48            DEC     AX
-p

AX=0000  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=073F  IP=0104   NV UP EI PL ZR NA PE NC
073F:0104 3D0000        CMP     AX,0000
-p

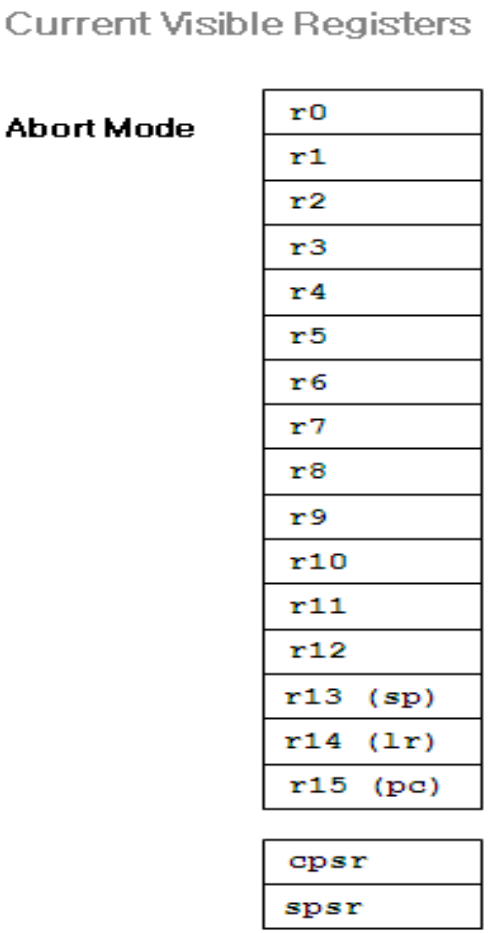
AX=0000  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=073F  IP=0107   NV UP EI PL ZR NA PE NC
073F:0107 75FA          JNZ     0103
-p

AX=0000  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=073F  IP=0109   NV UP EI PL ZR NA PE NC
073F:0109 90            NOP
```

2.1 CPU의 기본 구조

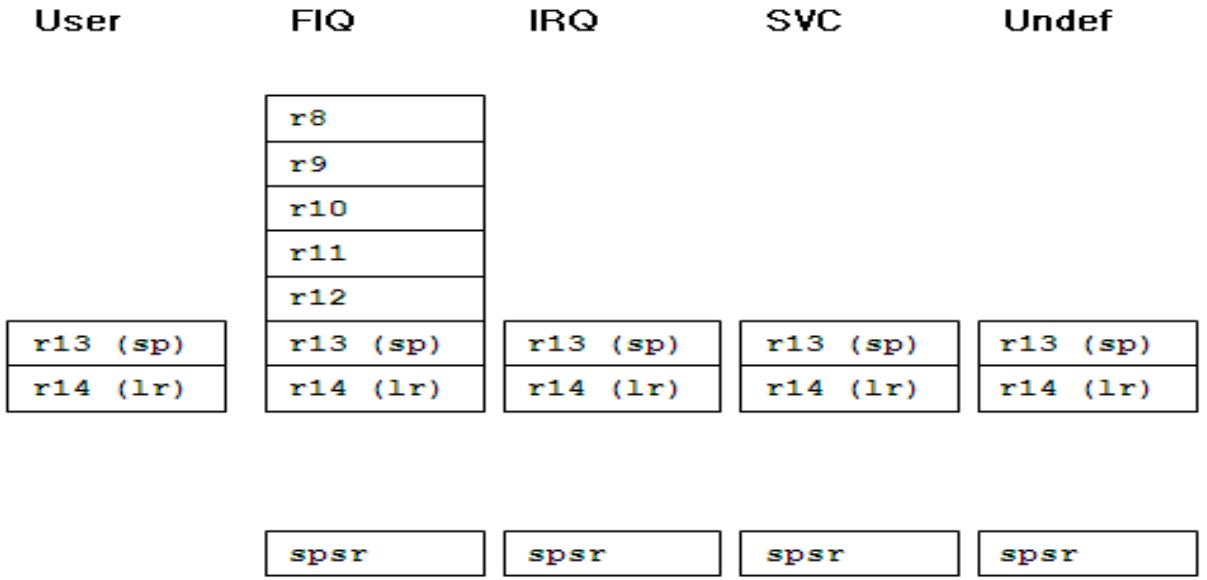
■ 참고: ARM(32비트)계열의 레지스터

✓ ARM 코어 버전 마다 다를 수 있다.



32비트 구조, word-32bit(버전마다 조금씩 다르다)

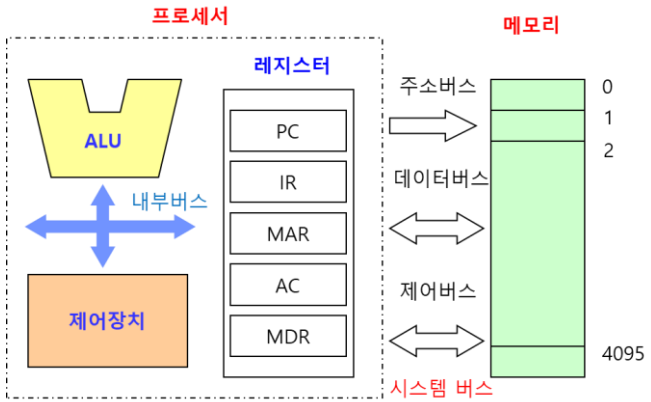
- 37개의 32비트 Register
- USER : 대부분의 태스크가 실행되는 비특권 모드
- FIQ : 높은 우선순위 인터럽트 발생시 진입
- IRQ : 낮은 우선순위 인터럽트 발생시 진입



2.1 CPU의 기본 구조

P63~p64

3) 제어 유니트 (제어 장치, Control Unit)



- ✓ 상세 내용은 제 4장 참조
- ✓ CPU 내에서 명령어의 처리를 총괄 제어하는 부분
  - ➢ **Brain within the Brain**

1) **명령어 인출**

2) **명령어의 해독** → 마이크로 프로그램(서브루틴)을 선택
   - ❖ 명령어에 지정된 주소지정방식에 따른 유효주소 계산 및 오퍼랜드 인출

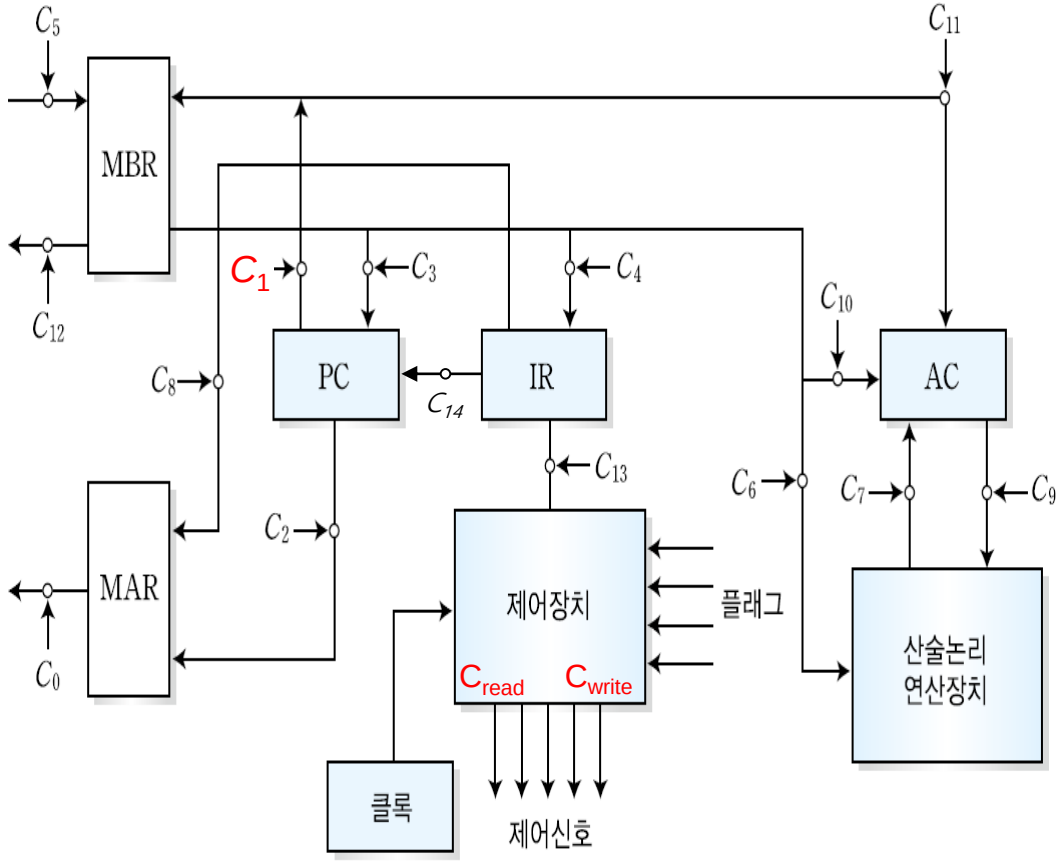
3) **마이크로 명령어의 실행 단계에 맞추어 클록(clock)과 동기하여 필요한 제어 신호들을 순차적으로 발생**
   - ❖ CPU 내부의 각각의 논리회로들을 순차적으로 구동 시켜 명령을 실행
   - ❖ 필요 시 CPU 외부로 제어 신호들을 순차적으로 내보낸다.
   - ❖ 명령어에 지정된 각 레지스터 입출력 실행
   - ❖ 명령어에 지정된 ALU에 대한 연산/입출력의 지정, 메모리 읽기 또는 쓰기 신호 발생

CPU에서 저장(레지스터), 산술논리 연산(ALU)을 제외한 모든 기능을 담당하며
레지스터, ALU도 제어 유니트의 지시에 따라서만 동작됨 → **모든 컴퓨터 동작을 관리**

2.1 CPU의 기본 구조

3) 제어 유니트 (제어 장치)

3) 마이크로 명령어의 실행 단계에 맞추어 클록(clock)과 동기하여 필요한 제어 신호들을 순차적으로 발생 - 계속



명령어 사이클	마이크로-연산	발생 제어신호
인출	t_0 : MAR ← PC	C_2
	t_1 : MBR ← M(MAR) PC ← PC + 1	C_5, C_{Read}
	t_2 : IR ← MBR	C_4
간접 (메모리 직접 주소지정 모드)	t_0 : MAR ← IR(Address)	C_8
	t_1 : MBR ← M(MAR)	C_5, C_{Read}
	t_2 : IR(Address) ← MBR(Address)	C_4
인터럽트	t_0 : MBR ← PC	C_1
	t_1 : MAR ← SP PC ← Routine-address	
	t_2 : M(MAR) ← MBR	C_{12}, C_{write}

2.1 CPU의 기본 구조

RISC와 CISC 방식 모두 인출단계는 유사하나
실행단계에서 하드웨어의 구성이 달라진다.

P63~p64

3) 제어 유니트의 구성 방식 → 4장 상세학습

✓ 순차적인 제어 신호를 발생 시키는 방법에 따라 2가지 중 하나로 CPU를 **설계**한다

1) 고정배선(Hard-wired) 제어방식

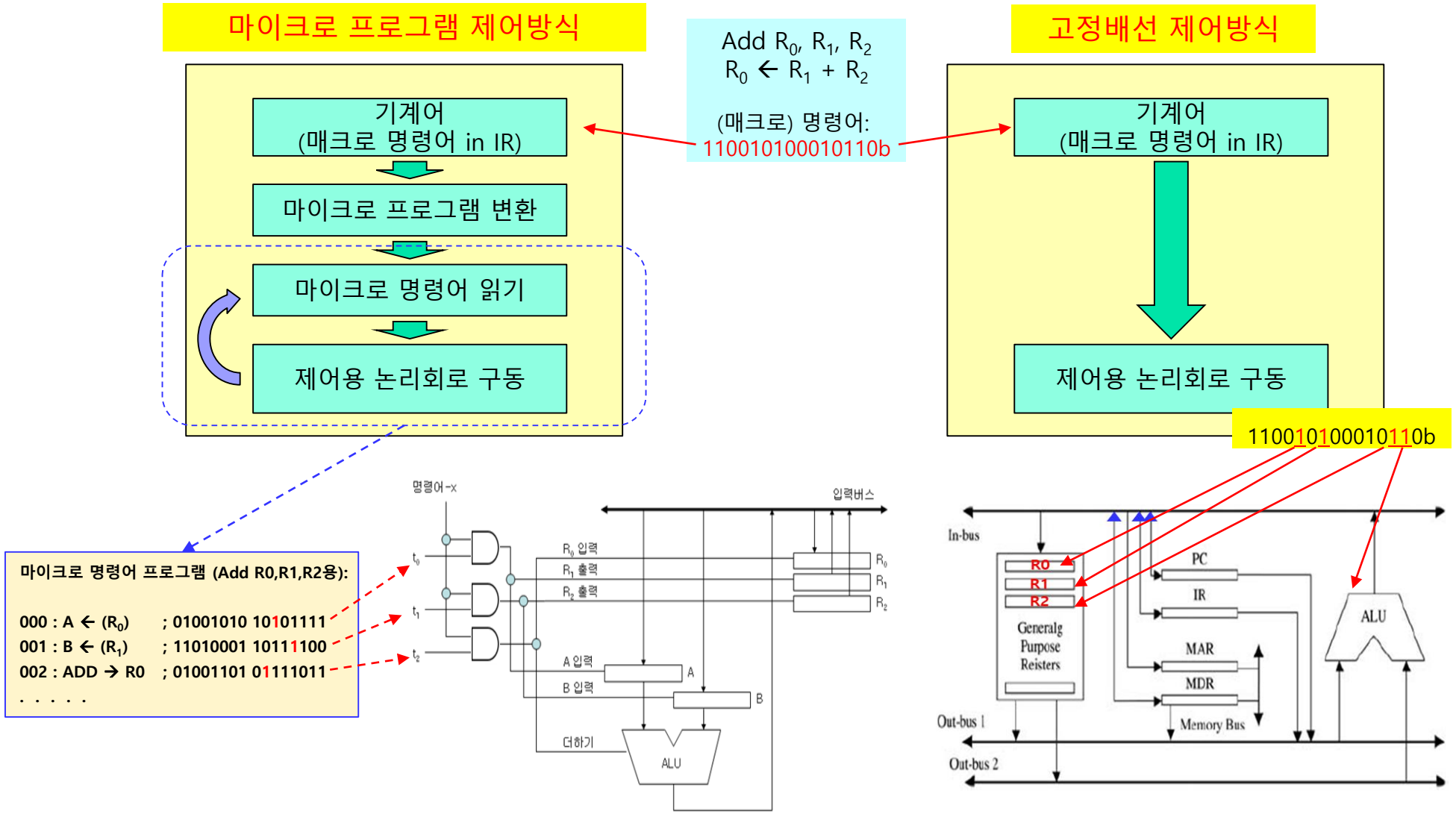
- 기계어 = (매크로)명령어 = 제어 신호 + 기타정보
- 일반적으로 RISC에서 사용 → 빠르다
- 입력된 (매크로)명령어를 바로 제어 신호의 출력 정보로 사용
  - ❖ 입력된 명령어의 일부 비트 정보가 제어용 논리회로를 구동 시키는 개별 입력 신호로 작용

2) 마이크로 프로그램(microprogrammed) 제어방식

- 기계어 = (매크로)명령어 = 마이크로 명령어 (제어 신호+기타정보)들의 조합
- 일반적으로 CISC에서 사용 → 속도는 늦지만 많은 명령어 표현 가능
- CPU 내부에는 마이크로 프로그램들이 저장되어 있는 제어용 ROM이 별도로 존재
  - ① 입력된 (매크로)명령어에 해당하는 **마이크로 프로그램**(마이크로 명령어들의 집합)을 찾아서
  - ② 마이크로 프로그램에서 구성한 **마이크로 명령어들**을 차례로 불러오고
  - ③ 이들 마이크로 명령어들의 일부 비트가 제어용 논리회로의 입력으로 사용
- 수평 마이크로 명령어 방식과 수직 마이크로 명령어 방식이 있다

2.1 CPU의 기본 구조

3) 제어 유니트의 2가지 구성 방식



2.1 CPU의 기본 구조

■ 추가: RISC와 CISC

1) CISC (Complex Instruction Set Computer) 프로세서

- 복합적인 연산들을 처리하는 다양한 종류의 명령어(기계어)들을 가진 기존의 프로세서 → 초기 CPU
- CISC의 문제점
  - ① 명령어 비트 수가 길어져서 기억장치로부터 읽어오는데 여러 번의 인출 필요
  - ② 다양한 종류와 길이의 명령어를 해독하기 위한 제어 유니트의 내부 회로 복잡도 증가
  - ③ 명령어 해독 시간 증가 (매크로 명령어를 이용해 해당되는 마이크로 명령어 루틴의 시작주소를 찾아야 함)
  - ④ 복수개의 마이크로 명령어를 처리하기 때문에 실행시간이 증가

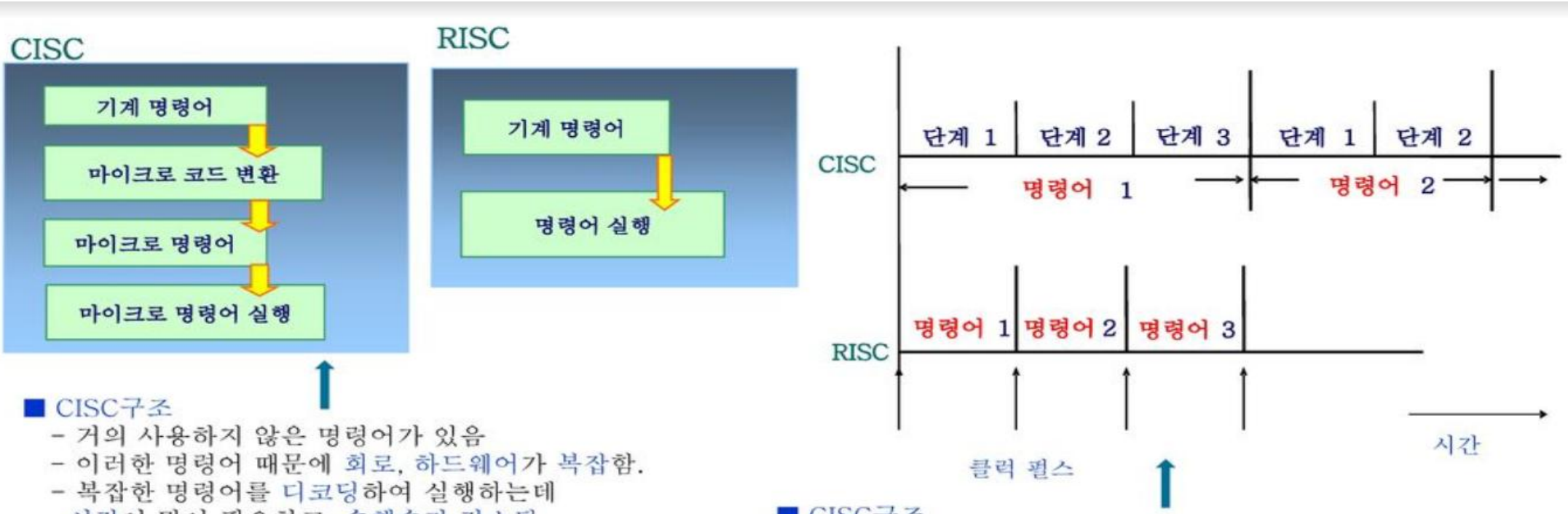
2) RISC (Reduced Instruction Set Computer) 프로세서

- CISC의 한계성을 극복하고자 제안한 새로운 프로세서 구조
- 간략화 된 소수의 명령어들만 지원하며 주로 고정배선 제어 유니트를 활용
- 고정된 길이의 명령어, 많은 레지스터들(주기억장치 접근 최소화)
- 빠른 실행속도를 지원하나 명령어가 제한적이어서 컴파일러가 매우 중요

근래에는 순수한 RISC/CISC CPU는 거의 없다. 서로의 장단점을 섞어 사용한다.

2.1 CPU의 기본 구조

■ 추가: RISC와 CISC



■ CISC구조
- 거의 사용하지 않은 명령어가 있음
- 이러한 명령어 때문에 회로, 하드웨어가 복잡함.
- 복잡한 명령어를 디코딩하여 실행하는데 시간이 많이 필요하고, 수행속가 감소됨.

■ RISC구조
- 자주 사용되고 아주 짧은 실행시간을 갖는 간단한 형태의 명령어들로 명령어 셋을 구성
- 마이크로 코드 변환 단계를 거치치 않기 때문에 빠르게 명령어를 실행
- 처리과정을 위해 CISC 컴파일러 보다 더 많은 명령어를 생성한다.

■ CISC구조
- 형태와 크기가 다양하므로 하나의 명령어를 실행하기 위하여 여러 단계를 거치게 되고 실행시간도 서로 다르다.

■ RISC구조
- 모든 명령어는 동일한 형태와 크기를 구성하고 있기 때문에 하나의 실행 주기 동안 하나의 명령어를 처리할 수 있다.

2.1 CPU의 기본 구조

■ 추가: RISC의 탄생 배경

✓ CISC(Complex Instruction Set computer) 프로세서의 명령어 집합의 확장 배경

- 컴퓨터가 다양한 분야에 사용되게 되었지만
  - 프로그램(S/W)의 크기가 증가
  - 메모리 가격이 고가 (S/W 라인 수가 적어야 한다)
  - 프로그램의 실행의 속도(CPU 클록 주파수)가 늦음
- 이에 대한 해결책으로 많은 명령어들을 CPU 내부에 내장
  - 300개 넘는 명령어 종류(거의 고급언어의 명령어와 유사한 기계어)가 사용되어 CPU H/W가 커짐
  - 명령어의 길이도 일정하지 않고 명령어의 인출, 해독, 실행 시에도 복잡하고 지연이 발생 → 제어블록이 너무 복잡해짐
- 따라서 새로운 개선책이 필요
  - 기존의 프로그램 사용에 대한 분석을 통한 보완책 수립 필요 → **RISC의 탄생 배경**
- RISC (Reduced Instruction Set computer) 프로세서의 출현
  - 간략화 된 소수의 명령어들만 지원하는 새로운 CPU 구조의 도입으로 다양한 장점을 지원

2.1 CPU의 기본 구조

■ 추가: RISC의 탄생 배경

1) 프로그램의 동작 분포 분석 (기존 CISC 문제점 해결을 위한 조사)

   ① 할당문과 같은 단순 데이터 이동이 복잡한 동작보다 중요

   ② 조건 분기가 아주 많다. 파이프라인 사용시 중요

   ③ 프로시저 호출자와 피호출자 간의 파라미터 전송이 최소의 명령어 실행으로 가능하도록 장치를 고안

   ④ 지역 변수의 저장과 접근하는 방식

동작(명령어)	%
할당문	35
루프	5
프로시저 호출	15
조건분기	40
무조건분기	3
기타	2

2) 분석을 통한 개선안

- 개선 방향 1 : 단순한 명령어를 빠르게 수행하는 구조 - ①, ②
  - 명령어 집합을 간단히 만들고 복잡하고 불필요한 명령어들은 제외. (복잡한 명령어 사용은 2% 이내)
  - 명령어 파이프라인의 설계를 최적화 시켜서 컴파일 된 코드의 실행 속도가 최소가 되도록 한다.
- 개선 방향 2 : 주기억장치 접근을 최소화하는 구조 - ③, ④
  - 많은 수의 레지스터를 사용해서 오퍼랜드 참조를 최적화해서 메모리 접근을 줄인다

2.1 CPU의 기본 구조

■ 추가: RISC의 특징 정리

1) 고정된 길이의 명령어
2) 제한된 개수의 명령어 (128 혹은 그 이하)
3) 제한되고 간단한 어드레싱(주소지정) 모드
4) 모든 동작이 레지스터에서 수행된다.
5) 메모리 동작은 적재와 저장의 2가지이다.
6) 파이프라인 명령어 실행
7) 많은 수의 범용 레지스터 혹은 레지스터 사용 최적화 위한 고급 컴파일러 기술
8) 클록 사이클당 하나의 명령어 실행
9) 제어장치 설계에 마이크로 프로그래밍 방식보다는 고정배선 방식을 사용

■ RISC와 CISC

구분	RISC	CISC
명령어	적음(고정길이)	많음(가변길이)
레지스터수	많음	적음
처리속도	빠름	느림
전력소모	적음	많음
가격	낮음	높음
용도	서버, 워크스테이션	개인용 PC
프로그래밍	복잡	단순

2.1 CPU의 기본 구조

P63~p64

4) CPU 내부 버스(Bus)

✓ 컴퓨터에서 다양한 부품들을 연결하는 전기 신호선들의 묶음

➢ 개별 신호선의 이름을 부르기에는 숫자가 너무 많다

➢ 따라서 이들을 묶어서 **버스**라고 간략히 부른다

✓ Bus의 세부 구성 — 컴퓨터에서 수는 0부터 시작

① 주소버스(Address Bus)

❖ 필요한 주소를 지정, 단방향의 주소관련 신호선들의 묶음

❖ 표기법 : A_{n-1} ~ A_0 (32개의 주소선 → A_{31}~A_0)

❖ **우리 교재에서는 하나의 기억장소에 바이트가 아닌 워드를 지정**

② 데이터버스(Data Bus)

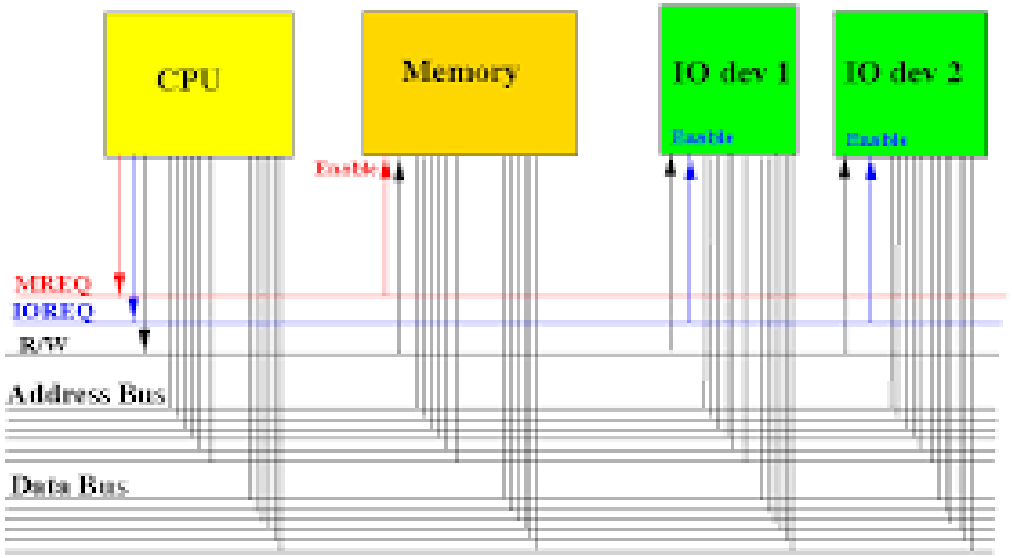
❖ 데이터 이동 통로, 양방향의 데이터 관련 신호선들의 묶음

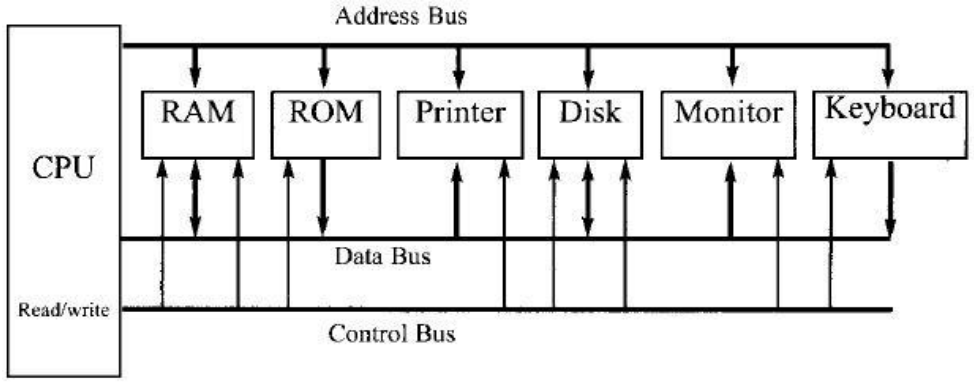
❖ 표기법 : D_{n-1} ~ D_0 (32개의 데이터선 → D_{31}~D_0)

③ 제어버스(Control Bus)

❖ 다양한 각종 제어 관련 신호선들의 묶음 (읽기, 쓰기, 인터럽트....)

❖ 표기법 : R/W!, IRQn .....





2.1 CPU의 기본 구조

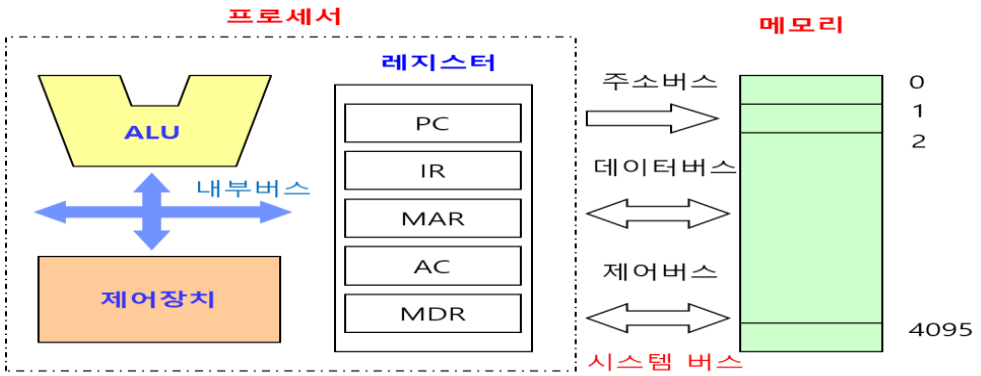
4) Bus의 종류(위치에 따라)

① CPU 내부 버스

- CPU 내부에서만 사용 (ALU, Register, 제어 유니트)
- 주소버스, 데이터버스, 제어버스로 구성
- 외부 부품 연결용 시스템 버스와는 직접 연결되지 않으며, 반드시 버퍼 레지스터 또는 시스템 버스 인터페이스를 통해 접속 된다.
  → MAR, MDR(MBR)

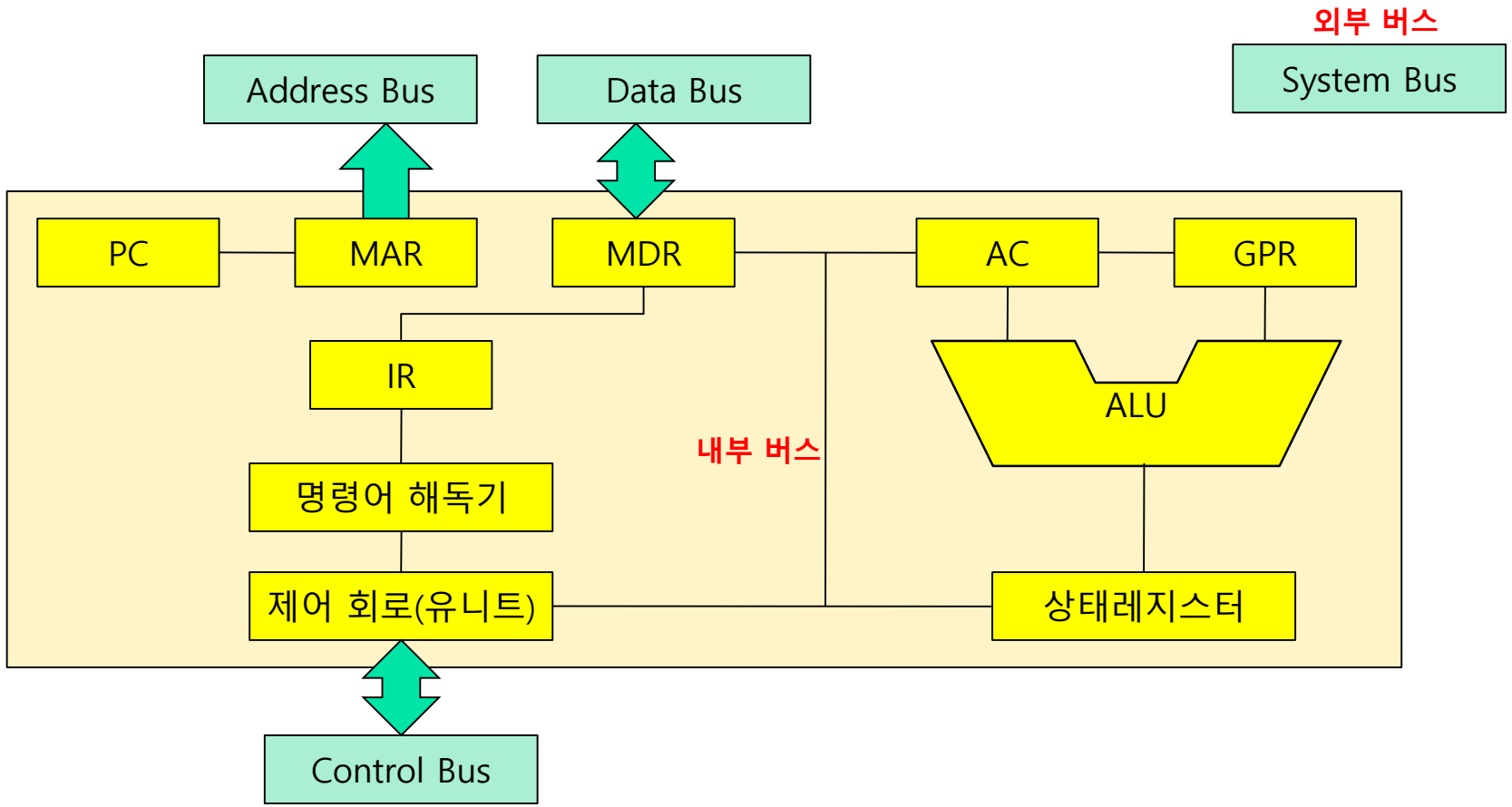
② 시스템 버스 (I/O 버스)

- CPU와 외부 부품과의 연결을 위해서 사용
- 주소버스, 데이터버스, 제어버스로 구성
- 용도, 속도 등에 따라 복수개의 다른 버스 구조로 구성하기도 한다. → 버스의 계층적 구조



2.1 CPU의 기본 구조

■ 참고 : CPU Block Diagram (정리)



2.2 명령어 실행

p65~p67

■ 명령어 사이클 (Instruction Cycle)

- ✓ **CPU가 한 개의 명령어를 실행하는데 필요한 전체 과정**
- ✓ CPU가 프로그램 실행을 시작한 순간부터 전원을 끄거나 회복 불가능한 오류가 발생하여 중단될 때까지 명령어 사이클을 반복 실행

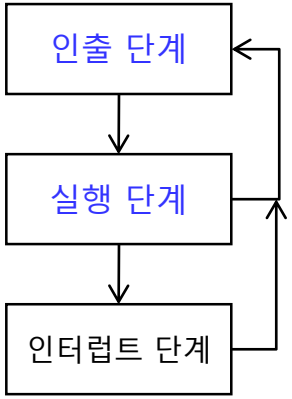
■ 명령어 부 사이클 (Sub-cycle)

- ✓ 한 개의 명령어를 실행하는 데 필요한 처리 과정을 세부 단계로 구분한 것
  - ➢ 파이프라인의 단계(stage)에 이용이 가능
- ✓ 이후 부(서브) 사이클을 줄여서 그냥 사이클이라고 표현함
- ✓ 우리 교재에서는 설명을 쉽게 하기 위해 2단계로 나누어 설명하고 있다
  1) 인출 사이클(fetch cycle)
     - ❖ CPU가 기억장치로부터 명령어를 읽어오는 단계
  2) 실행 사이클(execution cycle)
     - ❖ 명령어를 실행하는 단계

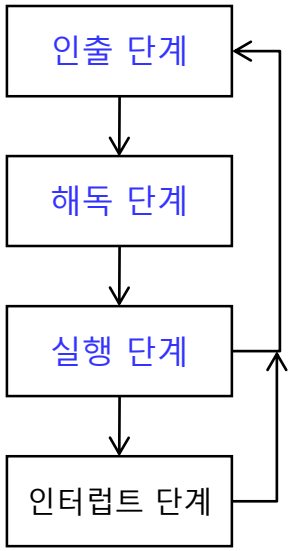
2.2 명령어 실행

■ 참고: 명령어 사이클 구분의 종류

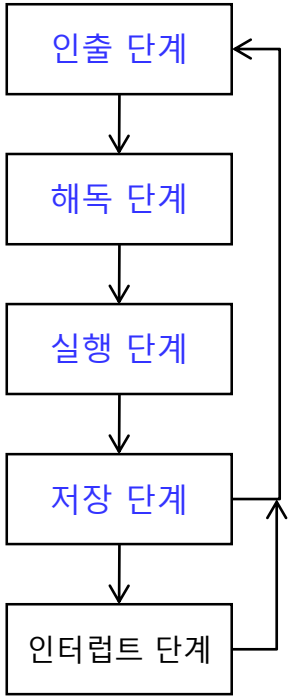
- ✓ 명령어 사이클을 구분 하는 방법은 다양한 종류가 있다
- ✓ 일반적으로 인터럽트 단계는 명령어 사이클 숫자 계산에서 제외하는 경우가 많음
- ✓ 정보처리기사 시험 등에서는 인터럽트 단계를 포함하여 그림 (d) 형태의 4-단계 명령어 사이클 구분법 사용



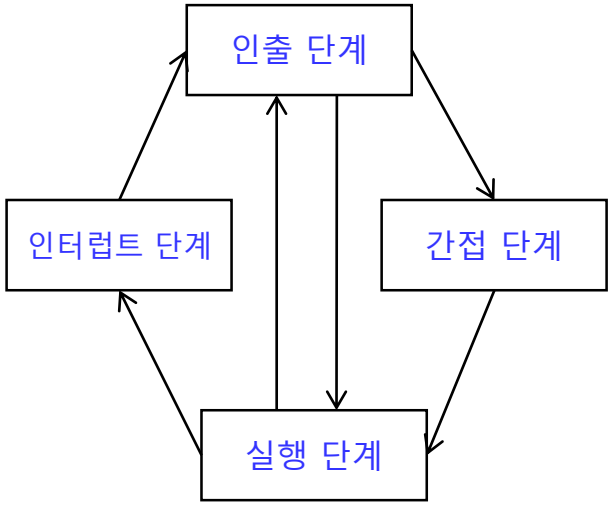
(a) 2-단계 명령어 사이클



(b) 3-단계 명령어 사이클



(c) 4-단계 명령어 사이클



(d) 4-단계 명령어 사이클

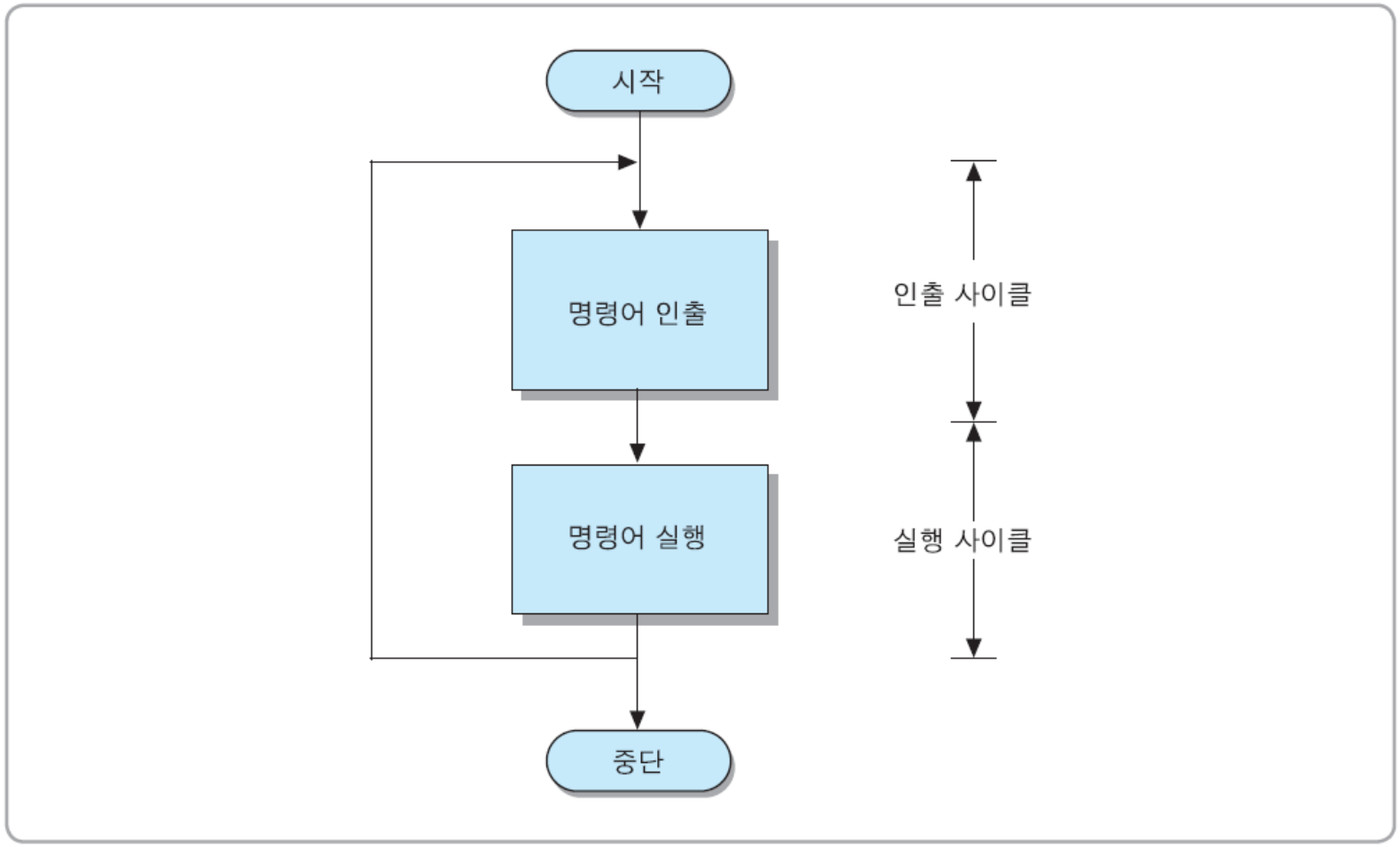
2.2 명령어 실행

p65~p67

■ 기본 명령어 사이클

✓ 전원이 꺼질 때까지 무한 반복

여기서는 설명을 쉽게 하기 위해 2단계 명령어 사이클로 설명

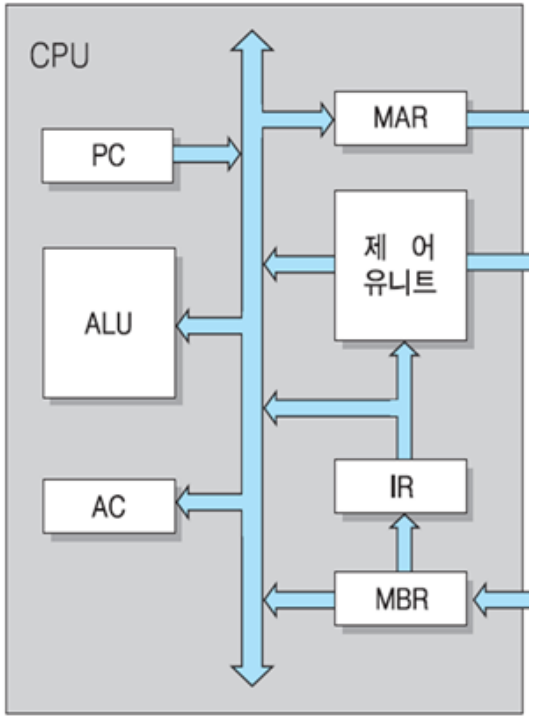


2.2 명령어 실행

p65~p67

■ 명령어 실행에 필요한 CPU 내부 레지스터들

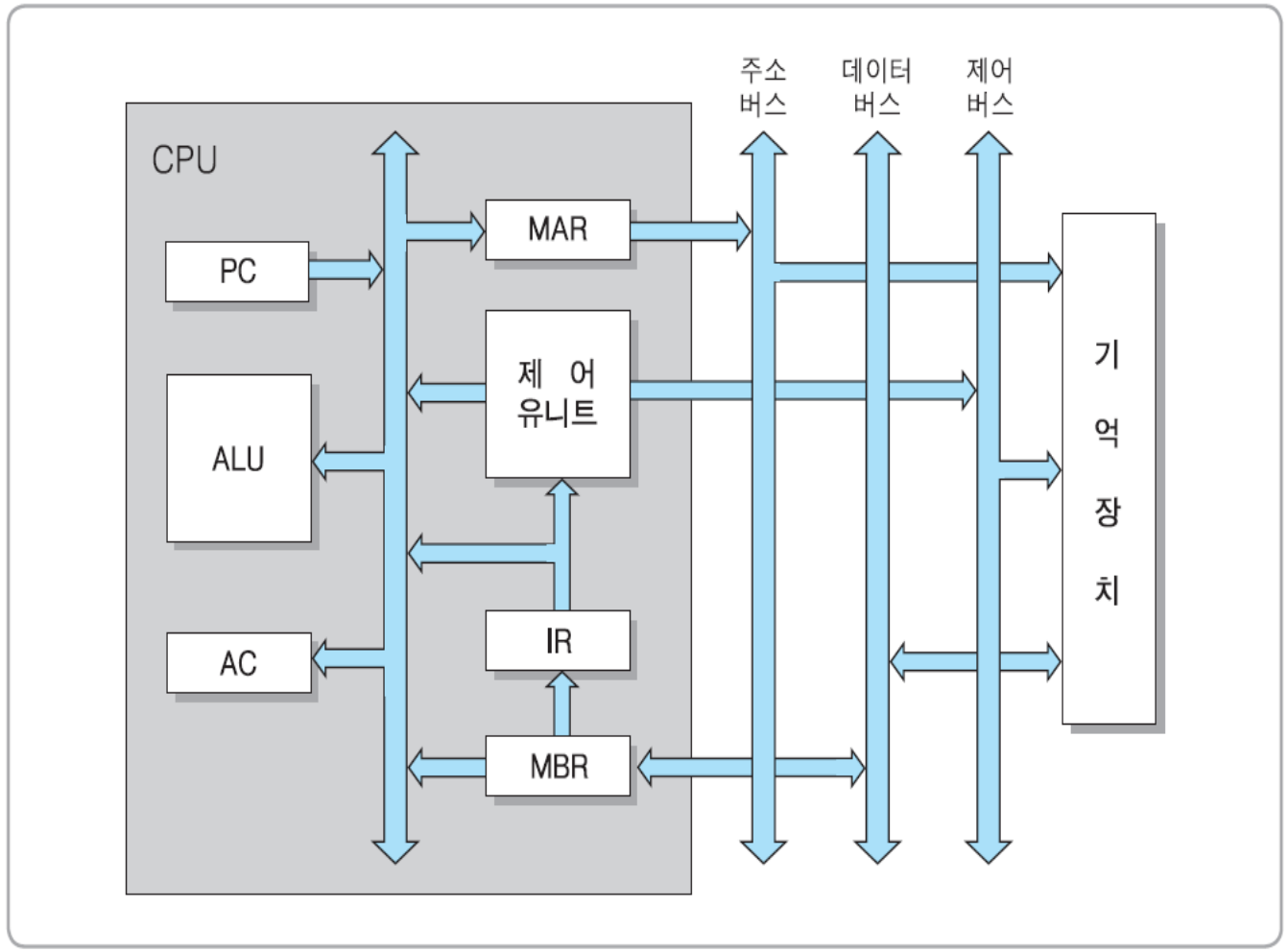
1) 프로그램 카운터(Program Counter: **PC**)
   - ➢ 제어 유니트가 다음에 인출할 명령어의 주소를 가지고 있는 레지스터
   - ➢ 각 명령어가 인출된 후에는 자동적으로 일정 크기(한 명령어 길이)만큼 증가
   - ➢ 분기(branch) 명령어가 실행되는 경우에는 목적지 주소로 갱신
2) 누산기(Accumulator: AC)
   - ➢ 데이터를 일시적으로 저장하는 레지스터레지스터의 길이는 CPU가 한 번에 처리할 수 있는 데이터 비트 수(단어 길이)와 동일
3) 명령어 레지스터(Instruction Register: **IR**)
   - ➢ 가장 최근에 인출된 명령어 코드가 저장되어 있는 레지스터
4) 기억장치 주소 레지스터(Memory Address Register: **MAR**)
   - ➢ CPU 외부의 주소를 접근할 때 접근할 주소를 기록하는 레지스터
   - ➢ 시스템 주소 버스로 동시에 출력됨
5) 기억장치 버퍼 레지스터(Memory Buffer Register: **MBR** = **MDR**)
   - ➢ 기억장치에 쓰여질 데이터 혹은 기억장치로부터 읽혀진 데이터를 일시적으로 저장하는 버퍼 레지스터
   - ➢ 시스템 데이터 버스와 연결되어 있음



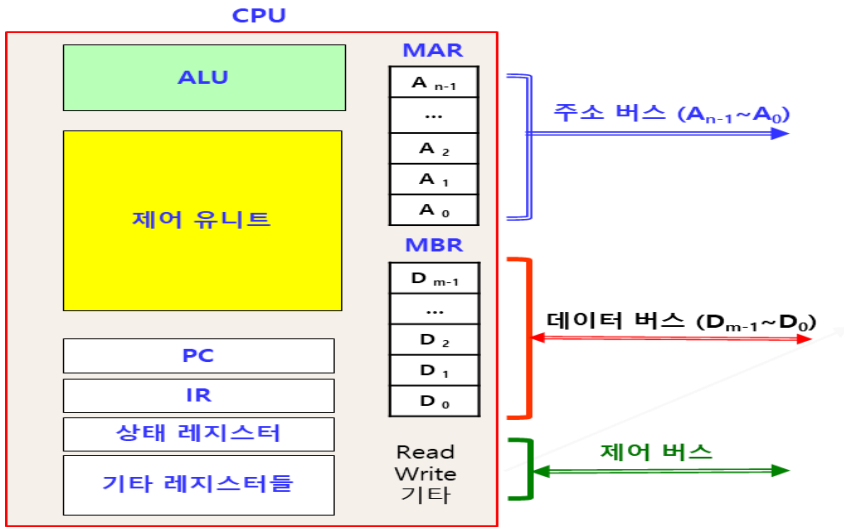
2.2 명령어 실행

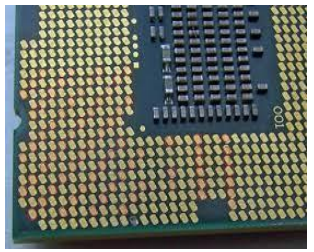
p65~p67

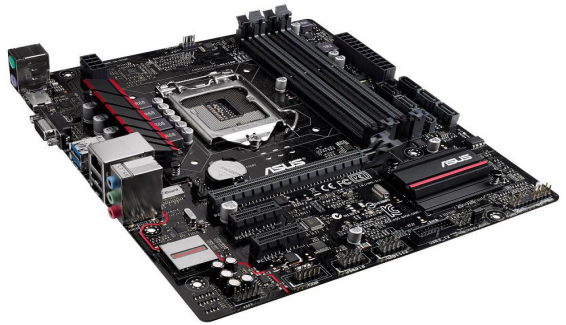
■ 데이터 통로(내부 버스)가 표시된 CPU 내부 구조



CPU는 시스템 버스의 주소, 데이터 버스와는 **MAR**, **MBR 레지스터**를 통해서만 연결된다







2.2 명령어 실행

■ CPU에서의 명령어 사이클의 실행 예

✓ 하나의 명령어를 읽고 실행하는 예

- ➢ 100번지의 명령어를 읽어 오고,
- ➢ 읽은 명령어가 'mov eax[300]' 인 경우 → 300번지 메모리의 내용을 내부 eax 레지스터로 읽어 오기

✓ 인출(반입) 단계

① PC 내용(100) → MAR → 시스템 주소버스 & 메모리 읽기 신호 출력

② 100번지의 메모리 내용(명령어) → 데이터 버스

③ 데이터 버스 → MBR → IR에 저장

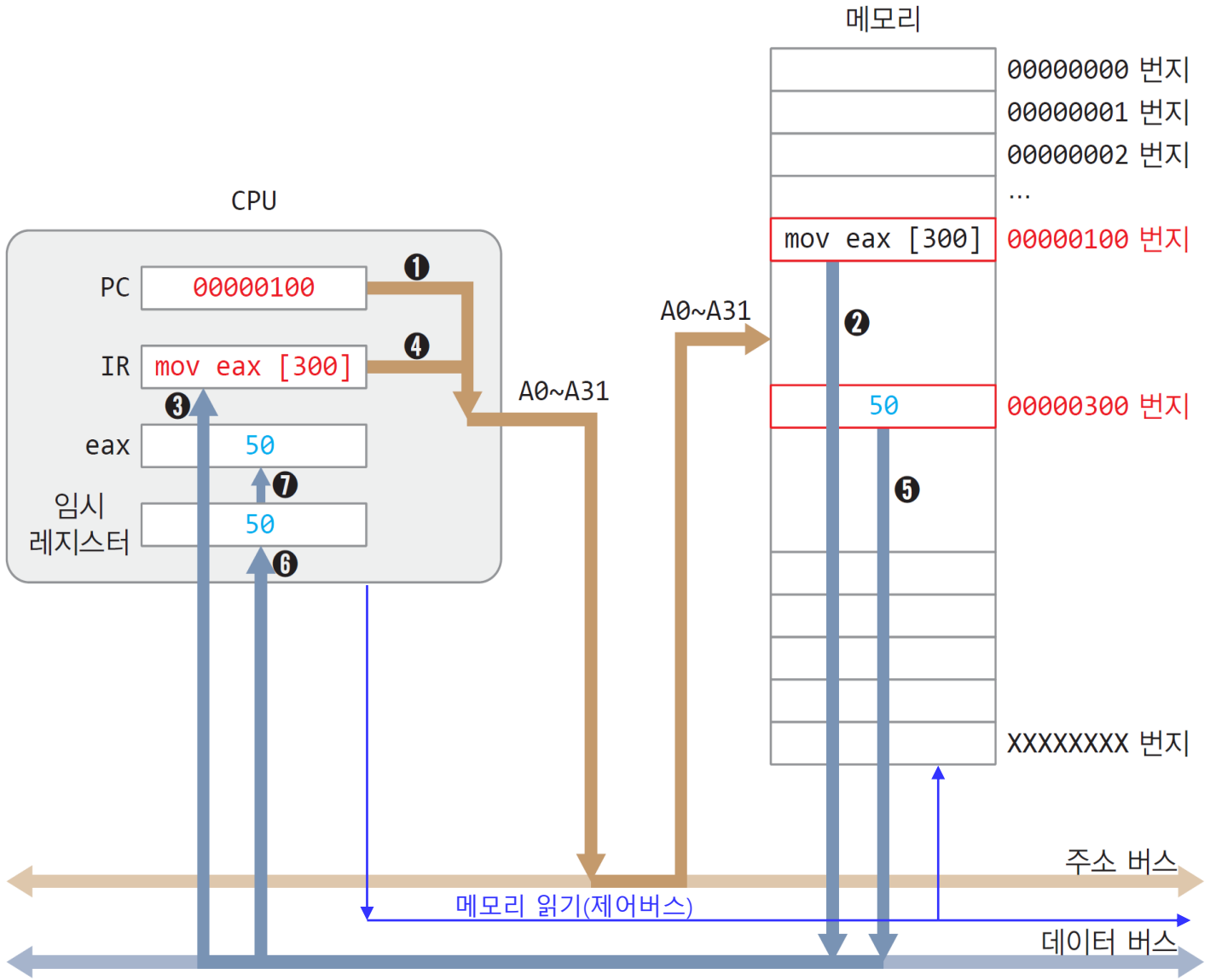
✓ 실행(수행) 단계

④ 명령어 분석 → 필요한 데이터 주소(300) → MAR → 시스템 주소버스 & 메모리 읽기 신호 출력

⑤ 300번지의 메모리(데이터 50) 내용 → 데이터 버스

⑥ 데이터 버스 → MBR → 임시 레지스터

⑦ 임시 레지스터 → eax



2.2.1 인출 사이클

p67~p69

■ 인출 사이클의 마이크로 연산(micro-operation)

① t_0 : MAR ← PC

② t_1 : MBR ← M[MAR], PC ← PC + 1

③ t_2 : IR ← MBR

✓ 단, t0, t1 및 t2 는 CPU 클록(clock)의 주기

우리 교재에서는 하나의 기억장소에 바이트가 아닌 워드를 지정

[예] CPU 클록 = 1GHz (클록 주기 = 1ns) 인 경우
인출 사이클에 소요되는 시간은 다음과 같다
→ 1ns x 3 = 3ns 소요

① 첫번째 주기 t_0

- 현재의 PC 내용을 CPU 내부 버스를 통하여 MAR로 전송

② 두번째 주기 t_1

- 그 주소가 지정하는 기억장치 위치로부터 읽혀진 명령어가 데이터 버스를 통하여 MBR로 적재되며,
- PC의 내용에 1을 더한다

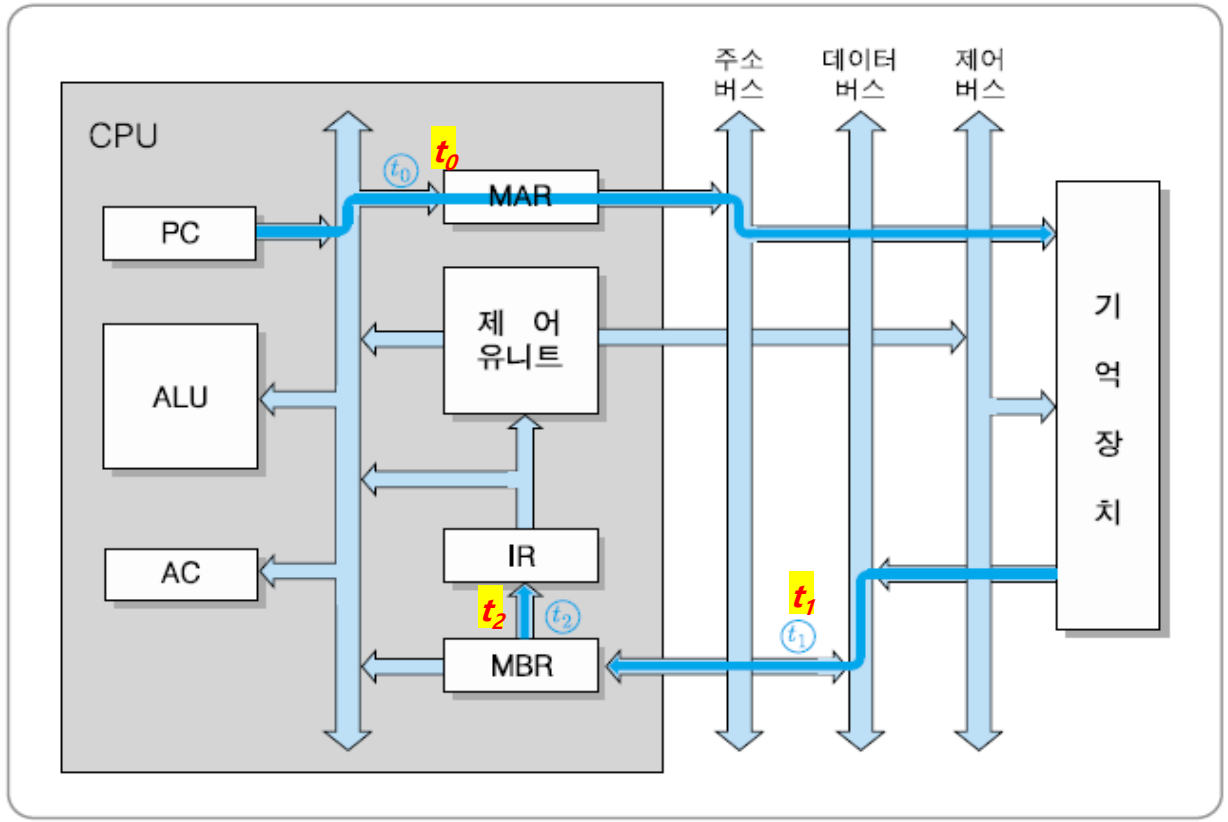
③ 세번째 주기 t_2

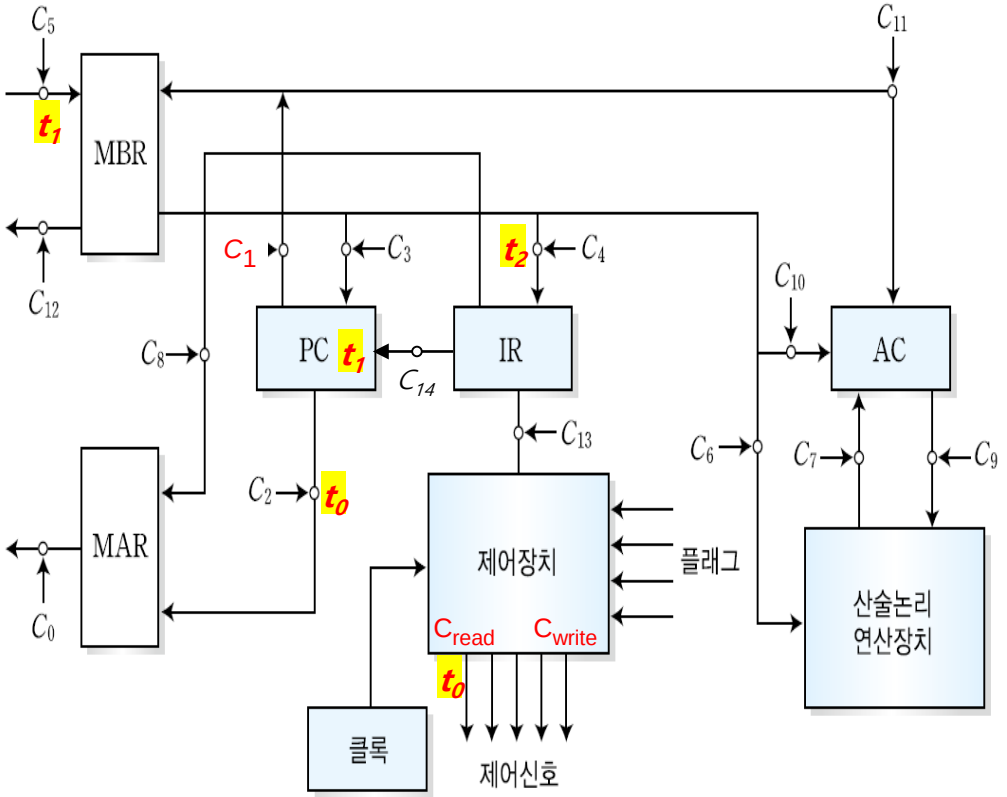
- MBR에 있는 명령어 코드가 명령어 레지스터인 IR로 이동

2.2.1 인출 사이클

p67~p69

- **인출 사이클의 주소 및 명령어 흐름도 (3개의 클록)**
  - ① t_0 : MAR ← PC, 메모리 읽기 제어 신호
  - ② T_1 : MBR ← M[MAR], PC ← PC + 1
  - ③ T_2 : IR ← MBR





2.2.2 실행 사이클

p67~p69

■ 인출 사이클의 주소 및 명령어 흐름도

✓ CPU는 실행 사이클 동안에 명령어 코드를 해독(decode)하고, 그 결과에 따라 필요한 연산들을 수행

✓ CPU가 수행하는 연산들의 종류

1) 데이터 이동
   - ❖ CPU와 기억장치 간 혹은 I/O장치 간에 데이터를 이동
2) 데이터 처리
   - ❖ 데이터에 대하여 산술 혹은 논리 연산을 수행
3) 데이터 저장
   - ❖ 연산 결과 데이터 혹은 입력장치로 부터 읽어 들인 데이터를 기억장치에 저장
4) 프로그램 제어
   - ❖ 프로그램의 실행 순서를 결정

✓ 실행 사이클에서 수행되는 마이크로-연산들은 명령어의 연산 코드(operation code= op code)에 따라 결정됨

2.2.2 실행 사이클

p69~p75

■ 기본적인 명령어 형식의 구성

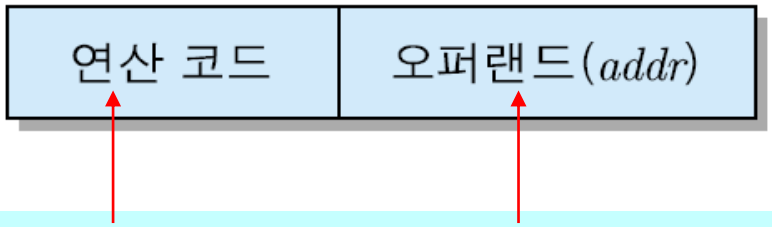
1) 연산 코드(operation code)

   ➢ CPU가 수행할 연산을 지정

2) 오퍼랜드(operand)

   ➢ 명령어 실행에 필요한 데이터 또는 데이터가 저장된 주소(addr)

   ➢ 연산 코드에 따라서 필드 구성이 달라진다.



[예] 더하기를 하는데 A와 B를 더해서 C에 저장하라

[예] 데이터 이동을 하는데 A에서 B로 이동하라

2.2.2 실행 사이클

p69~p75

■ [사례 1] LOAD *addr* 명령어

✓ 기억장치에 저장되어 있는 데이터를 CPU 내부 레지스터인 AC로 이동하는 명령어

① t_0 : MAR ← IR(*addr*), 메모리 읽기 제어 신호

② t_1 : MBR ← M[MAR]

③ t_2 : AC ← MBR

① 첫번째 주기 t_0

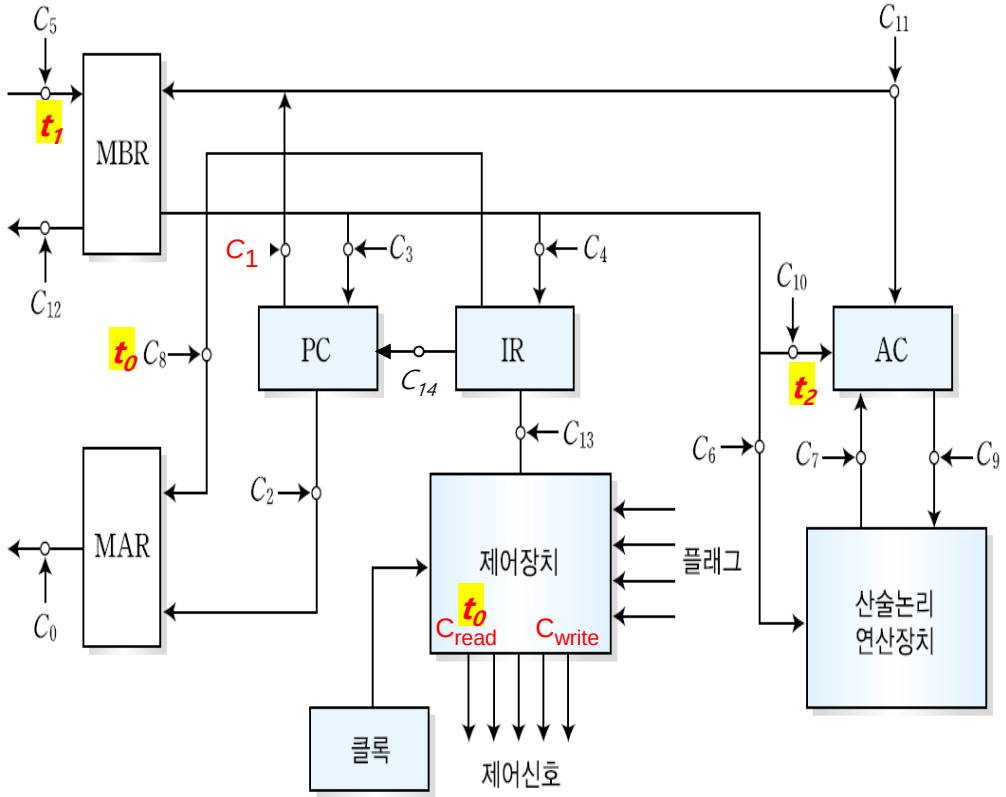
- IR에 있는 명령어의 주소 부분(오퍼랜드)을 MAR로 전송
- 메모리 읽기 제어 신호 출력

② 두번째 주기 t_1

- 그 주소가 지정한 기억장소로부터 데이터를 인출하여 MBR로 전송

③ 세번째 주기 t_2

- MBR에 있는 데이터를 AC에 적재



2.2.2 실행 사이클

p69~p75

■ [사례 2] STA *addr* 명령어

✓ AC 레지스터의 내용을 기억장치에 저장하는 명령어

① t_0 : MAR ← IR(*addr*)

② t_1 : MBR ← AC

③ t_2 : M[MAR] ← MBR, 메모리 쓰기 신호

① 첫번째 주기 t_0

➢ 데이터를 저장할 기억장치의 주소(IR 오퍼랜드)를 MAR로 전송

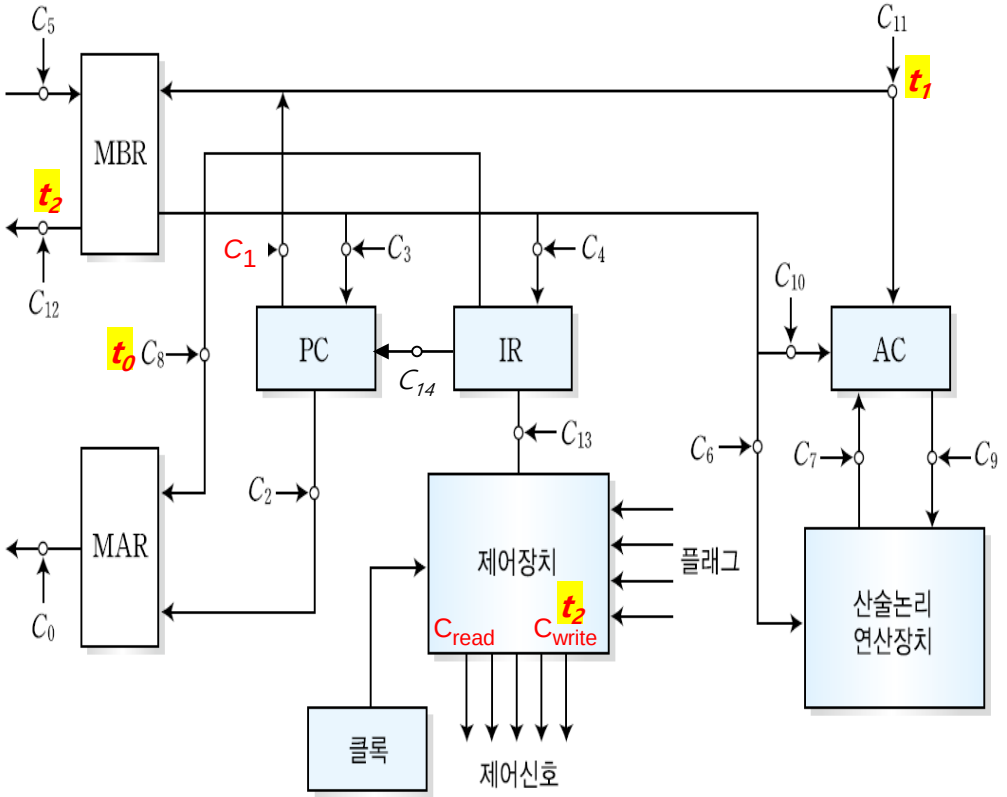
② 두번째 주기 t_1

➢ 저장할 데이터를 버퍼 레지스터인 MBR로 이동

③ 세번째 주기 t_2

➢ 메모리 쓰기 제어 신호 출력

➢ MBR의 내용을 MAR이 지정하는 기억장소에 저장



2.2.2 실행 사이클

p69~p75

■ [사례 3] ADD *addr* 명령어

✓ 기억장치에 저장된 데이터를 AC의 내용과 더하고, 그 결과는 다시 AC에 저장하는 명령어

① t_0 : MAR ← IR(*addr*), 메모리 읽기 제어 신호

② t_1 : MBR ← M[MAR]

③ t_2 : AC ← AC + MBR

① 첫번째 주기 t_0

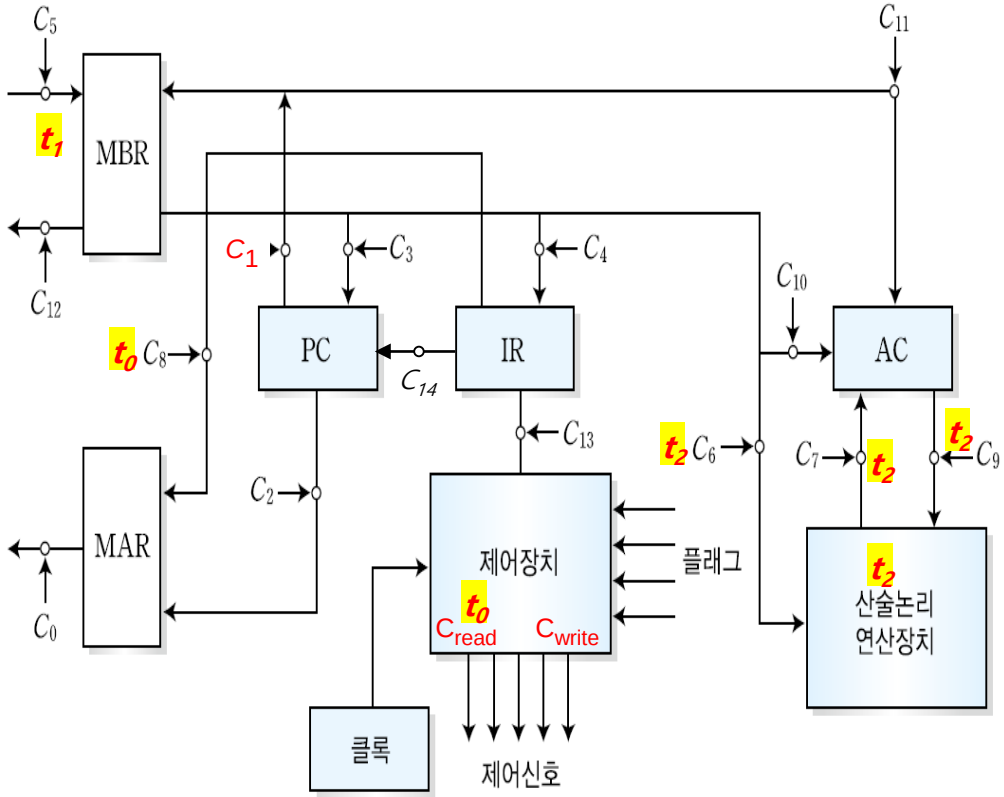
- 데이터를 읽어올 기억장치의 주소(IR 오퍼랜드)를 MAR로 전송
- 메모리 읽기 제어 신호 출력

② 두번째 주기 t_1

- 저장할 데이터를 버퍼 레지스터인 MBR로 이동

③ 세번째 주기 t_2

- 그 데이터와 AC의 내용을 더하고 결과값을 다시 AC에 저장



2.2.2 실행 사이클

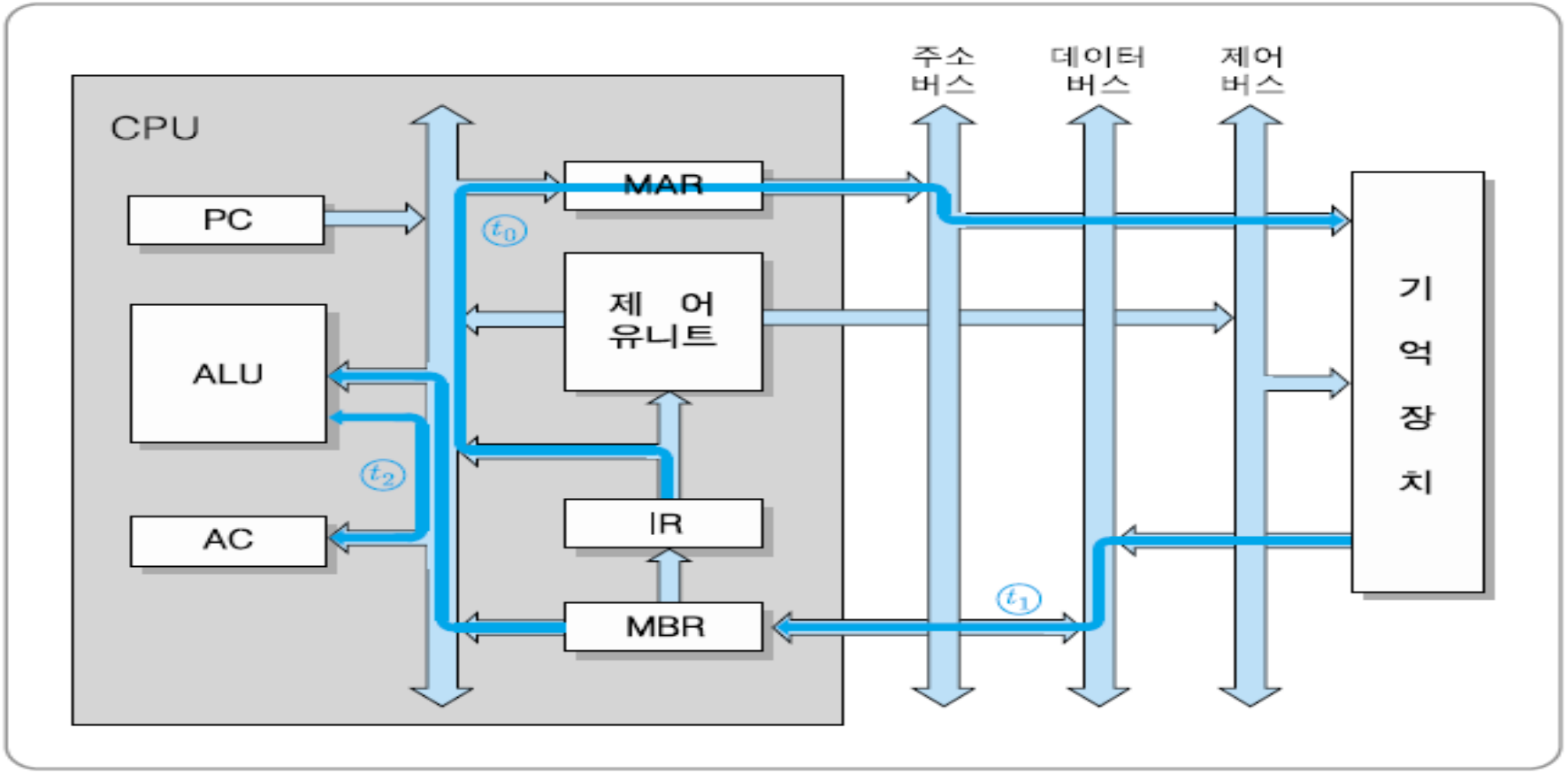
p69~p75

■ ADD 명령어 실행 사이클 동안의 정보 흐름

① t_0 : MAR ← IR(*addr*), 메모리 읽기 제어 신호

② t_1 : MBR ← M[MAR]

③ t_2 : AC ← AC + MBR



2.2.2 실행 사이클

p69~p75

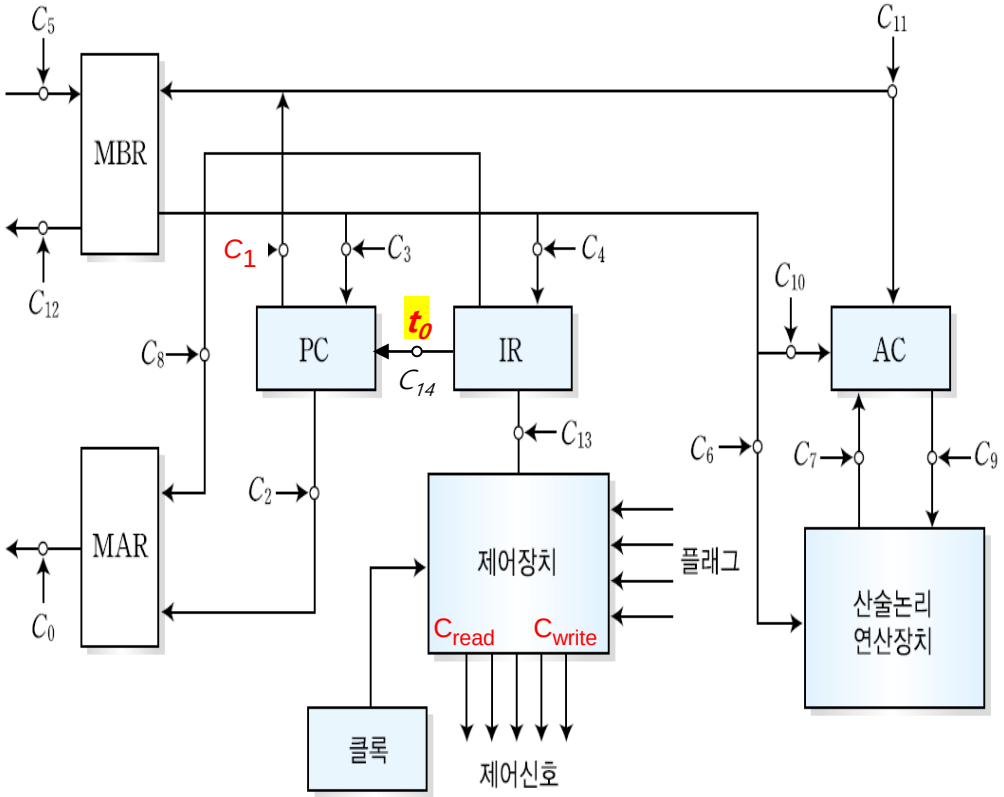
■ [사례 4] JUMP *addr* 명령어

✓ 오퍼랜드(addr)가 가리키는 위치의 명령어로 실행 순서를 변경하는 분기(branch) 명령어

① t_0 : PC ← IR(*addr*)

① 첫번째 주기 t_0

- 명령어의 오퍼랜드(분기할 목적지 주소)를 PC에 저장
- 다음 명령어 인출 사이클에서 그 주소의 명령어가 인출되므로 분기가 발생



2.2.2 실행 사이클

p69~p75

■ 어셈블리 프로그램 실행과정의 예

- ✓ 16 비트 CPU 이며 주소는 16비트(1워드 = 2바이트) 단위로 구할 경우
- ✓ 다음과 같이 명령어 연산 코드(4 비트)를 설정

  LOAD : 1

  STORE : 2

  ADD : 5

  JUMP : 8

- ✓ 나머지 3 자리(니블)는 주소(*addr*)로 사용
- ✓ 어셈블리 프로그램의 예

주소	명령어	기계 코드
100	LOAD 250	1250
101	ADD 251	5251
102	STA 251	2251
103	JUMP 170	8170

2.2.2 실행 사이클

p69~p75

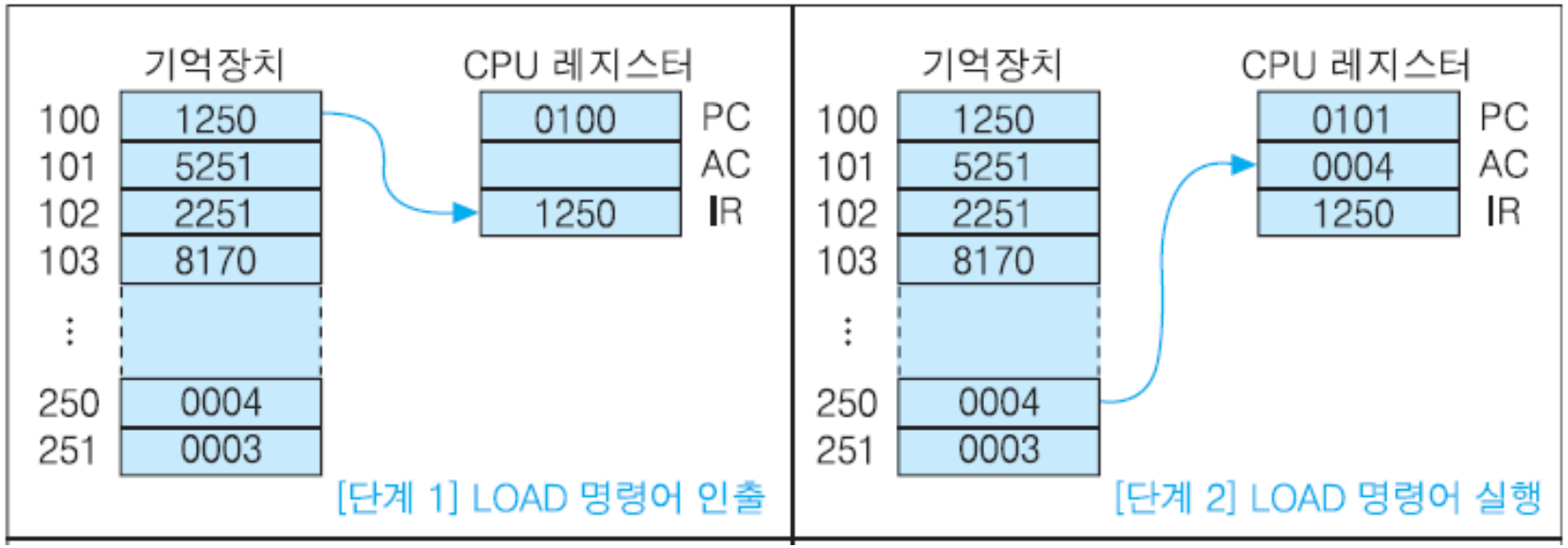
■ 어셈블리 프로그램 실행과정의 예 - 계속

① 100 번지의 첫 번째 명령어 코드가 인출되어 IR에 저장

② 250 번지의 데이터를 AC로 이동

③ PC = PC + 1 = 101 → 자동으로 변경

< 연산 코드 >
- LOAD : 1
- STORE : 2
- ADD : 5
- JUMP : 8



2.2.2 실행 사이클

p69~p75

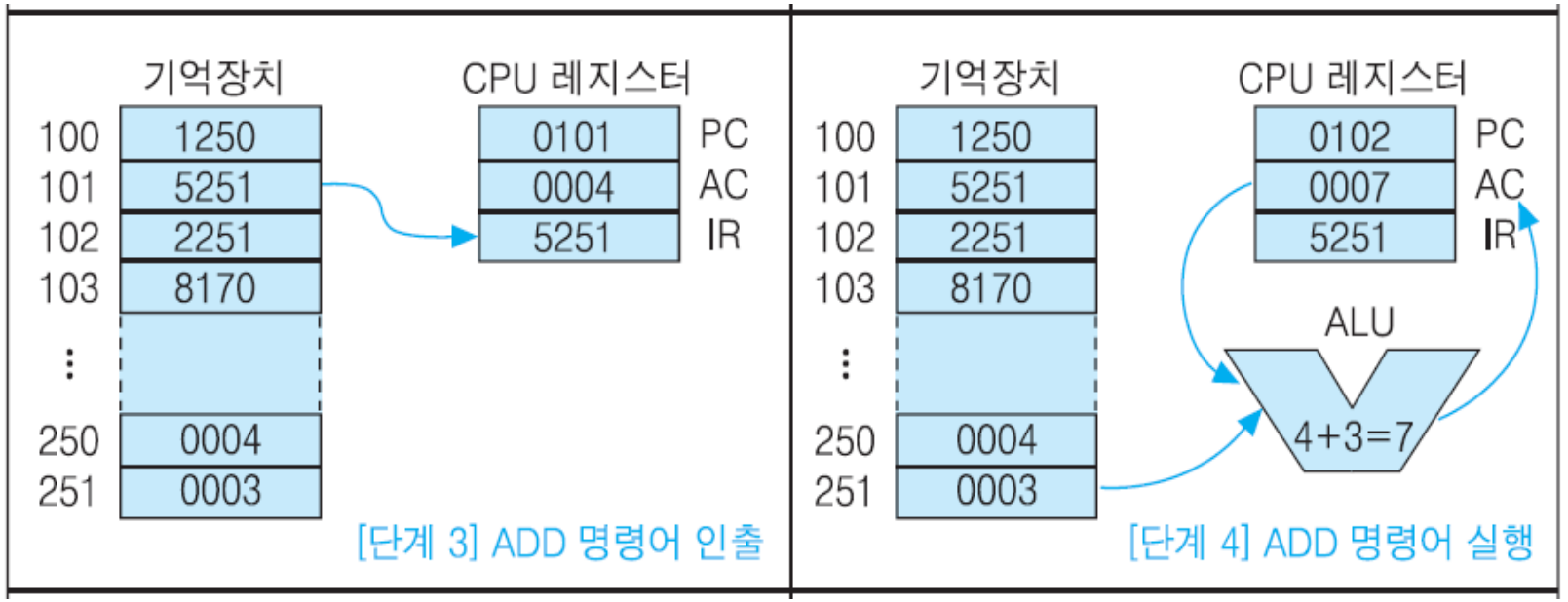
■ 어셈블리 프로그램 실행과정의 예 - 계속

④ 두 번째 명령어가 101 번지로부터 인출되어 IR에 저장

⑤ AC의 내용과 251 번지의 내용을 더하고, 결과를 AC에 저장

⑥ PC의 내용은 102로 증가

< 연산 코드 >
- LOAD : 1
- STORE : 2
- ADD : 5
- JUMP : 8



2.2.2 실행 사이클

p69~p75

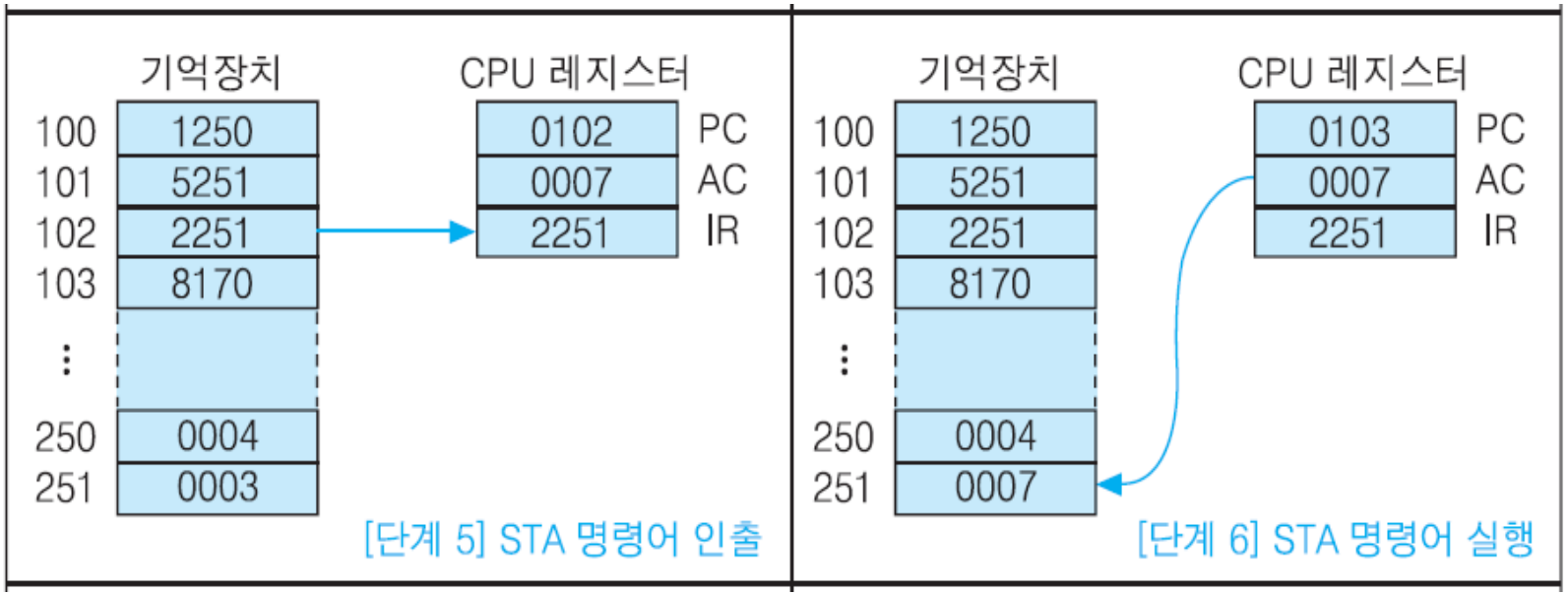
■ 어셈블리 프로그램 실행과정의 예 - 계속

⑦ 세 번째 명령어가 102 번지로부터 인출되어 IR에 저장

⑧ AC의 내용이 251 번지에 저장

⑨ PC의 내용은 103으로 증가

< 연산 코드 >
- LOAD : 1
- STORE : 2
- ADD : 5
- JUMP : 8



2.2.2 실행 사이클

p69~p75

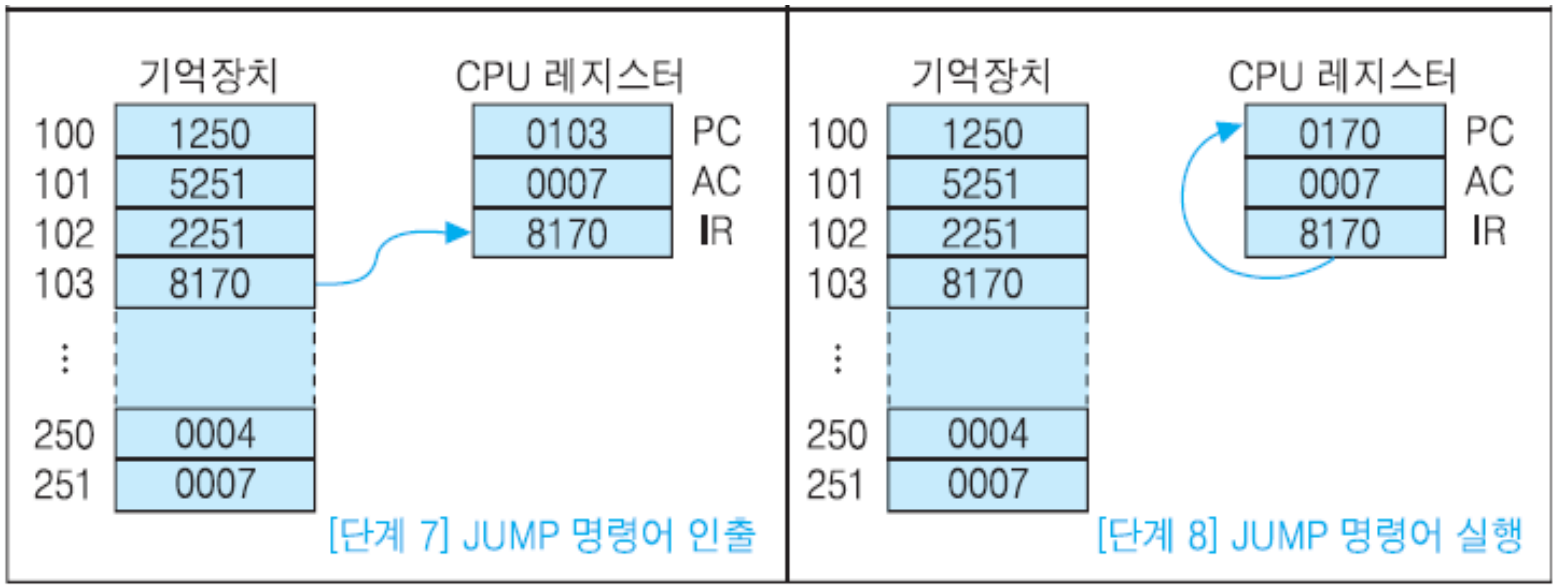
■ 어셈블리 프로그램 실행과정의 예 - 계속

⑩ 네 번째 명령어가 103 번지로부터 인출되어 IR에 저장

⑪ 분기될 목적지 주소, 즉 IR의 하위 부분(170)이 PC로 적재

➢ 다음 명령어 인출 사이클에서는 170 번지의 명령어를 인출

< 연산 코드 >
- LOAD : 1
- STORE : 2
- ADD : 5
- JUMP : 8

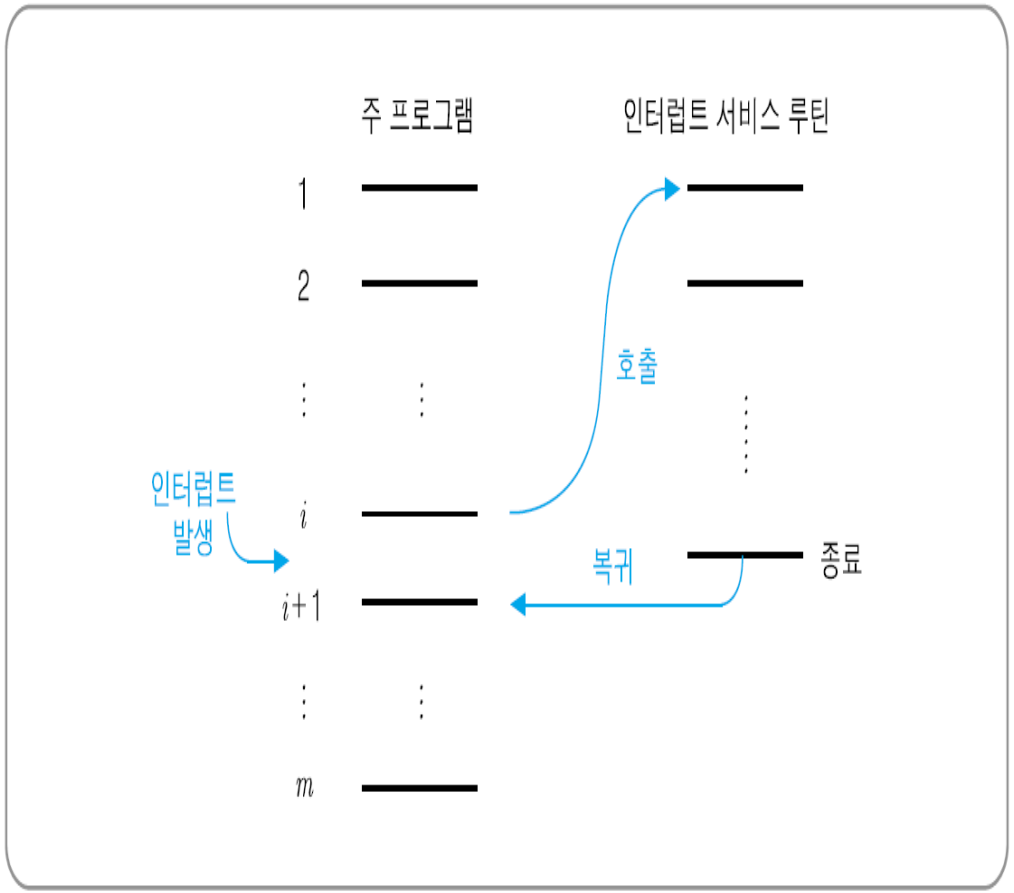


2.2.3 인터럽트 사이클(interrupt cycle)

p75~p80

■ 인터럽트

- ✓ 프로그램 실행 중에 CPU의 현재 처리 순서를 중단시키고 다른 동작을 수행하도록 요구하는 시스템 동작
- ✓ **CPU는 외부장치의 상황을 지속적으로 확인할 필요가 없어서 매우 효율적인 방식 →** 모든 CPU에서 사용 중
- ✓ 인터럽트 제어기(Interrupt Controller)가 필요
- ✓ 외부로부터 인터럽트 요구가 들어오면,
  - ① CPU는 원래의 프로그램 수행을 중단하고,
  - ② 요구된 인터럽트를 위한 인터럽트 서비스 프로그램을 먼저 수행
- ✓ 인터럽트 서비스 루틴(Interrupt Service Routine: **ISR**)
  - ➢ 인터럽트를 처리하기 위하여 수행되는 프로그램 루틴
  - ➢ 키보드, 타이머, 네트워크, 프린터 등 개별 인터럽트 마다 ISR 들은 처리하는 내용들이 모두 다름



2.2.3 인터럽트 사이클(interrupt cycle)

p75~p80

■ 인터럽트 처리 과정

✓ 인터럽트가 들어왔을 때 CPU는

① 어떤 장치가 인터럽트를 요구했는지 확인하고

② 해당 장치용 ISR을 호출하여 실행

③ 서비스가 종료된 다음에는 중단되었던 원래 프로그램의 수행 계속

✓ CPU 인터럽트 처리의 세부 동작 → 자세한 사항은 7.4절에서 설명

① 현재의 명령어 실행을 끝낸 즉시 인터럽트 요청이 있으면, 다음에 실행할 명령어의 주소(PC의 내용)를 스택(stack)에 저장

❖ 스택의 초기 값은 주기억장치의 특정 부분에 프로그램(운영체제)이 설정

② 실행해야 할 ISR을 호출하기 위하여 해당 ISR 루틴의 시작 주소를 PC에 적재

❖ 이때 시작 주소는 인터럽트를 요구한 장치로부터 전송되거나 미리 정해진 값으로 결정

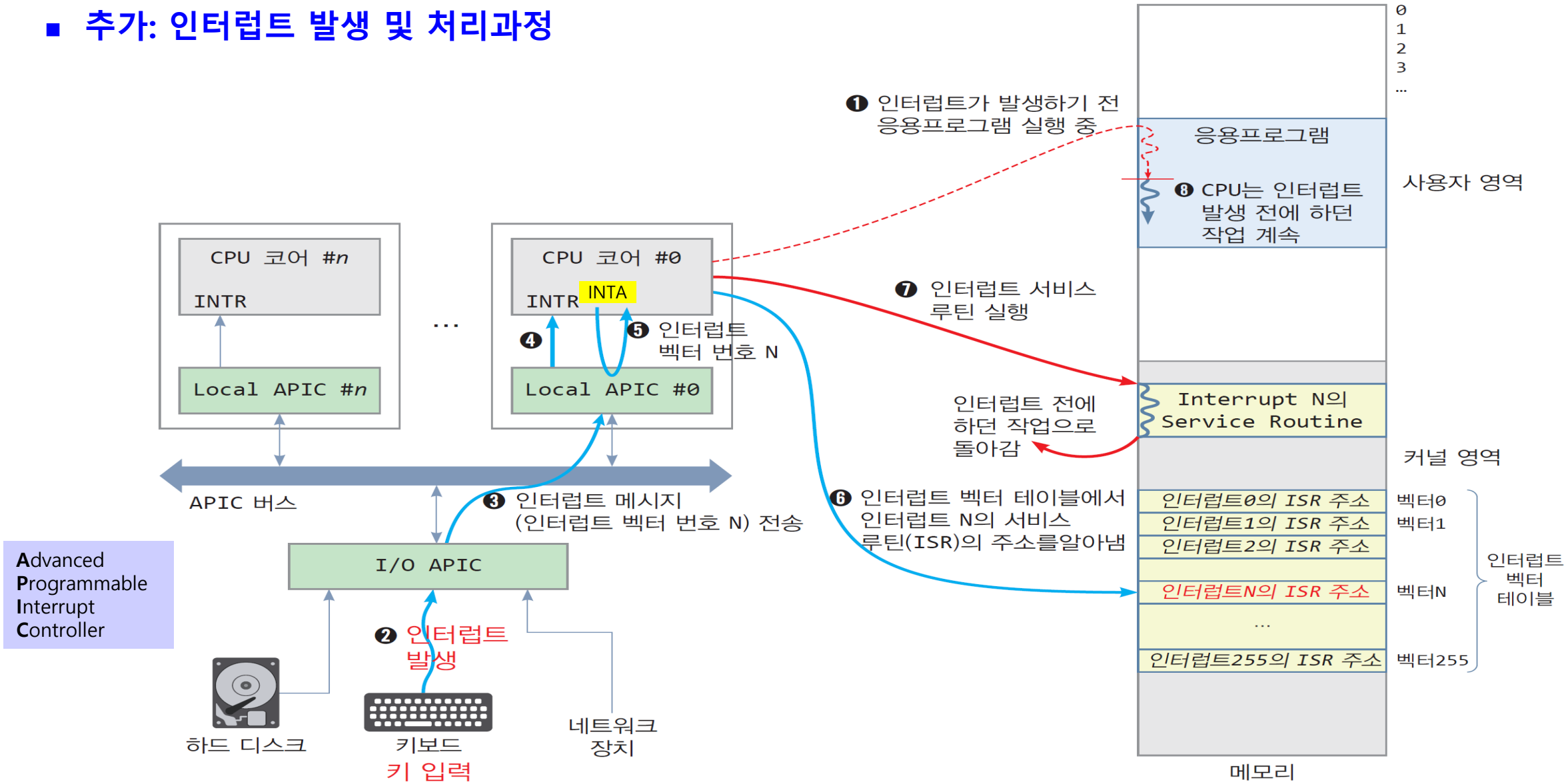
③ 해당 ISR을 실행

❖ ISR 내부에서 사용할 레지스터들을 스택에 저장 → ISR 실행 → 스택에 저장된 원래 프로그램용 레지스터들을 복원 → IRET

④ 스택에 저장된 원래 프로그램의 PC 값을 PC로 다시 저장해, 다음 클록의 입력시에 원래의 프로그램을 계속 실행

2.2.3 인터럽트 사이클(interrupt cycle)

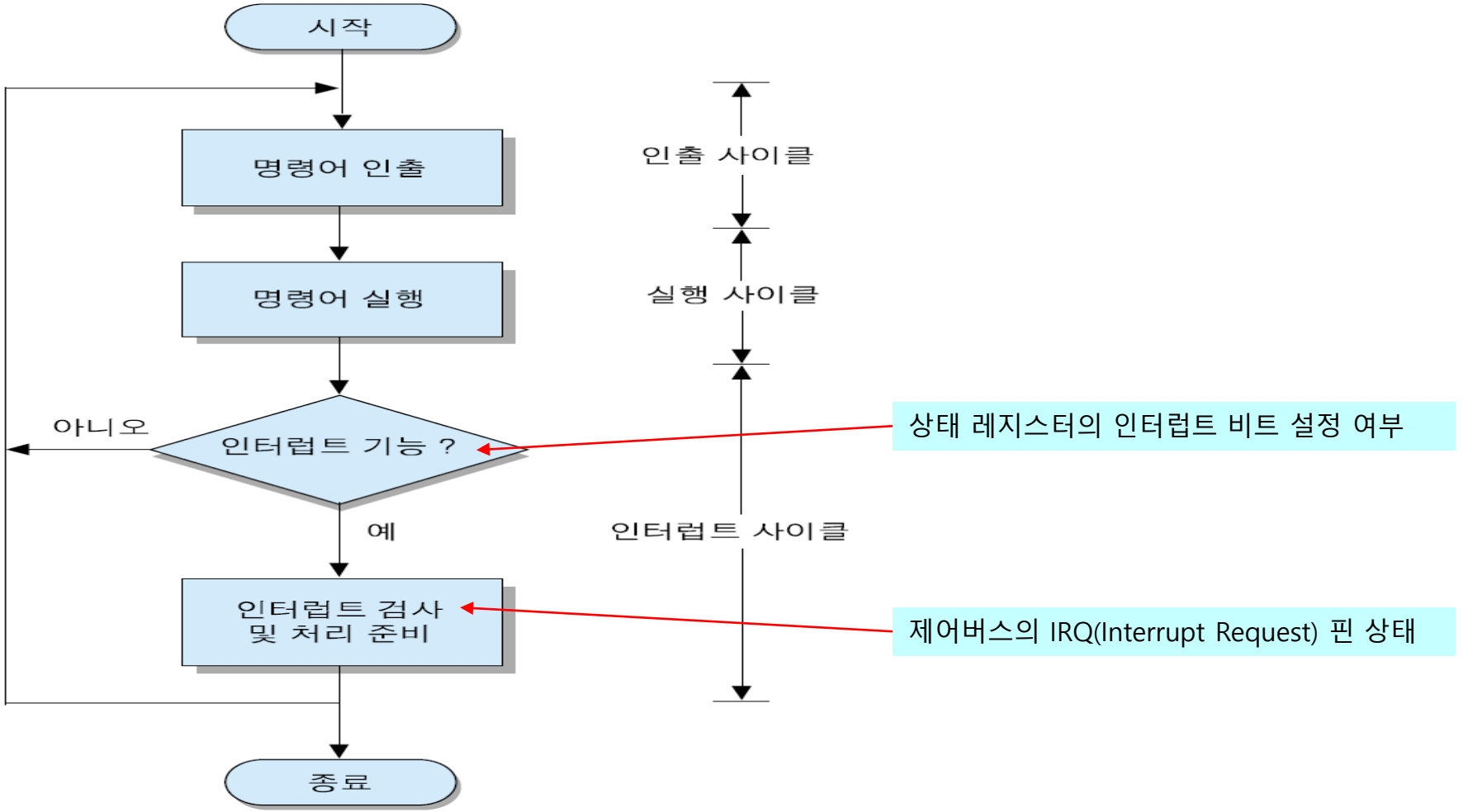
■ 추가: 인터럽트 발생 및 처리과정



2.2.3 인터럽트 사이클(interrupt cycle)

p75~p80

■ 인터럽트 사이클이 추가된 명령어 사이클



2.2.3 인터럽트 사이클(interrupt cycle)

p75~p80

■ 인터럽트 사이클의 마이크로 연산

① t_0 : MBR ← PC

② t_1 : MAR ← SP, PC ← ISR의 시작 주소

③ t_2 : M[MAR] ← MBR, 메모리 쓰기 제어 신호, SP ← SP-1

➢ 단, SP는 스택 포인터(stack pointer).

① 첫번째 주기 t_0

➢ PC의 내용을 MBR로 전송

② 두번째 주기 t_1 (그림에 없음)

➢ SP의 내용을 MAR로 전송하고

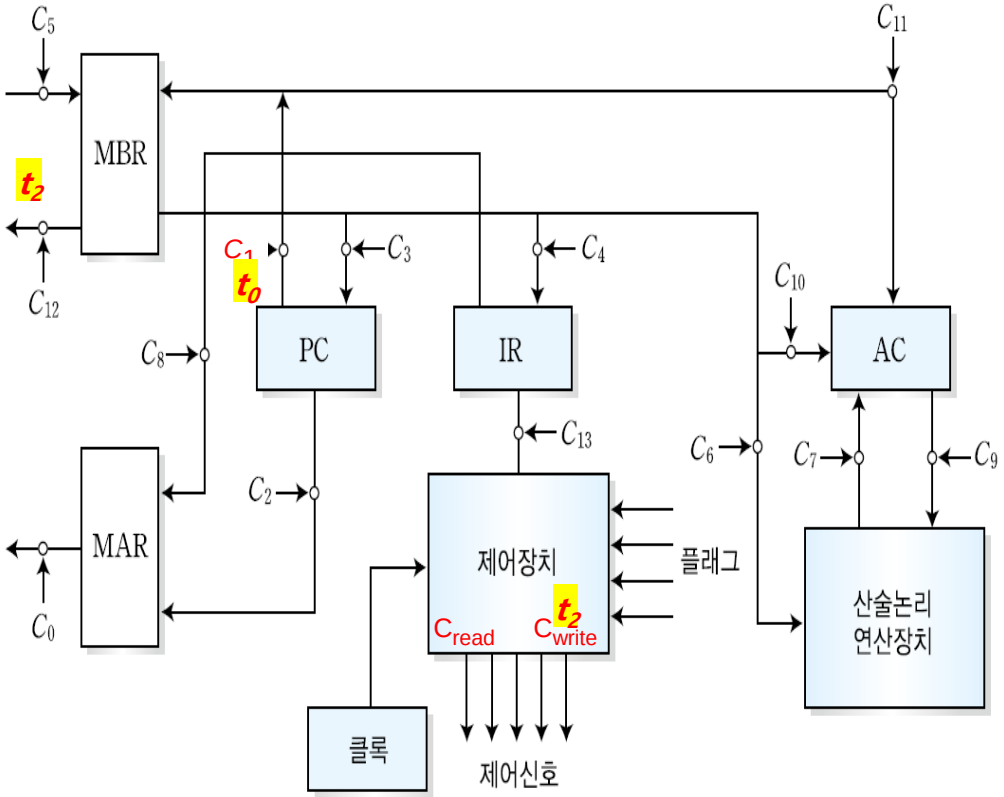
➢ PC의 내용은 인터럽트 서비스 루틴의 시작 주소로 변경

③ 세번째 주기 t_2

➢ 메모리 쓰기 제어 신호 출력

➢ MBR에 저장되어 있던 원래 PC의 내용을 스택에 저장

➢ SP 값을 조정(그림에 없음)



2.2.3 인터럽트 사이클(interrupt cycle)

p75~p80

■ 인터럽트 사이클의 마이크로 연산 [예]

- ✓ 아래 프로그램의 첫 번째 명령어인 LOAD 250 명령어가 실행되는 동안에 인터럽트가 들어왔으며
- ✓ 현재 SP = 999이고, 인터럽트 서비스 루틴의 시작 주소는 650 번지라고 가정

```
100    LOAD   250
101    ADD    251
102    STA    251
103    JUMP   170
```

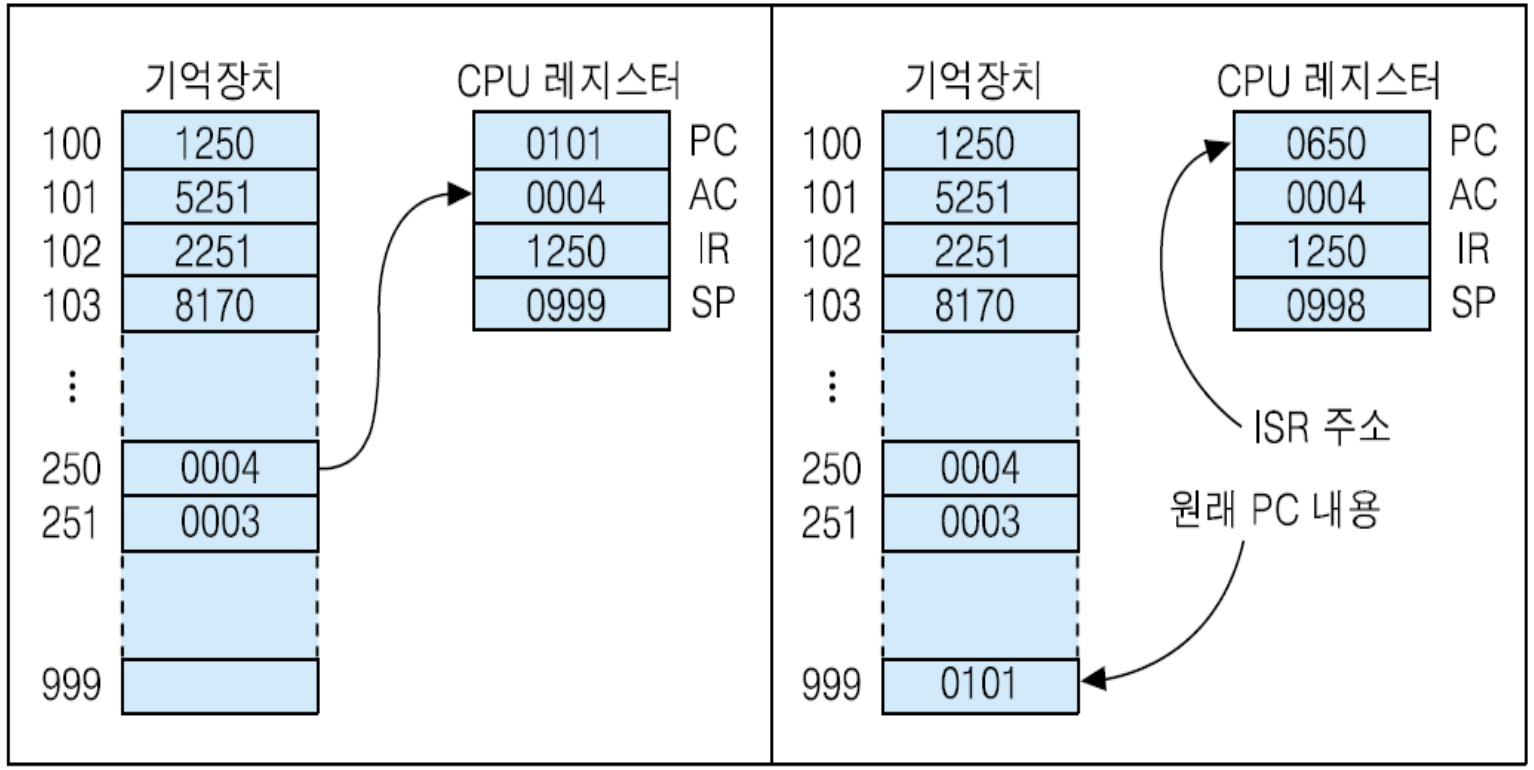
< 연산 코드 >
- LOAD : 1
- STORE : 2
- ADD : 5
- JUMP : 8

2.2.3 인터럽트 사이클(interrupt cycle)

p75~p80

■ 인터럽트 요구가 들어온 경우의 상태 변화

< 연산 코드 >
- LOAD : 1
- STORE : 2
- ADD : 5
- JUMP : 8



(a) LOAD 명령어의 실행 사이클이 종료된 상태 (b) 인터럽트 사이클이 종료된 상태

2.2.3 인터럽트 사이클(interrupt cycle)

p75~p80

■ 다중 인터럽트(multiple interrupt)

- ✓ 일반적으로 Nested Interrupt라고 부름
- ✓ 인터럽트 서비스 루틴을 수행하는 동안에 다른 인터럽트 발생 → 다중 인터럽트
- ✓ 다중 인터럽트의 처리방법 (두 가지)

1) CPU가 인터럽트 서비스 루틴을 처리하고 있는 도중에는 새로운 인터럽트 요구가 들어오더라도 인터럽트 사이클을 수행하지 않는 방법

   ❖ 시스템 운영상 중요한 프로그램이나 도중에 중단할 수 없는 데이터 입출력 동작 등을 위한 인터럽트를 처리하는데 사용

   ❖ 인터럽트 플래그(interrupt flag) ← 0

   ☞ 인터럽트 무시(interrupt disabled) 상태

2) 인터럽트의 우선순위(Priority)를 정하고, 우선순위가 낮은 인터럽트가 처리되고 있는 동안에 우선순위가 더 높은 인터럽트가 들어온다면, 현재의 인터럽트 서비스 루틴의 수행을 중단하고 새로운 인터럽트를 처리

< CPU 상태 레지스터의 예 (8086) >

IE	SV		OV	P	C	Z	S

조건 플래그
- 부호 (S, sign flag)
- 제로 (Z, zero flag)
- 자리올림수 (C, carry flag)
- 패리티 (P, parity flag)
- 오버플로우 (OV, overflag flag)

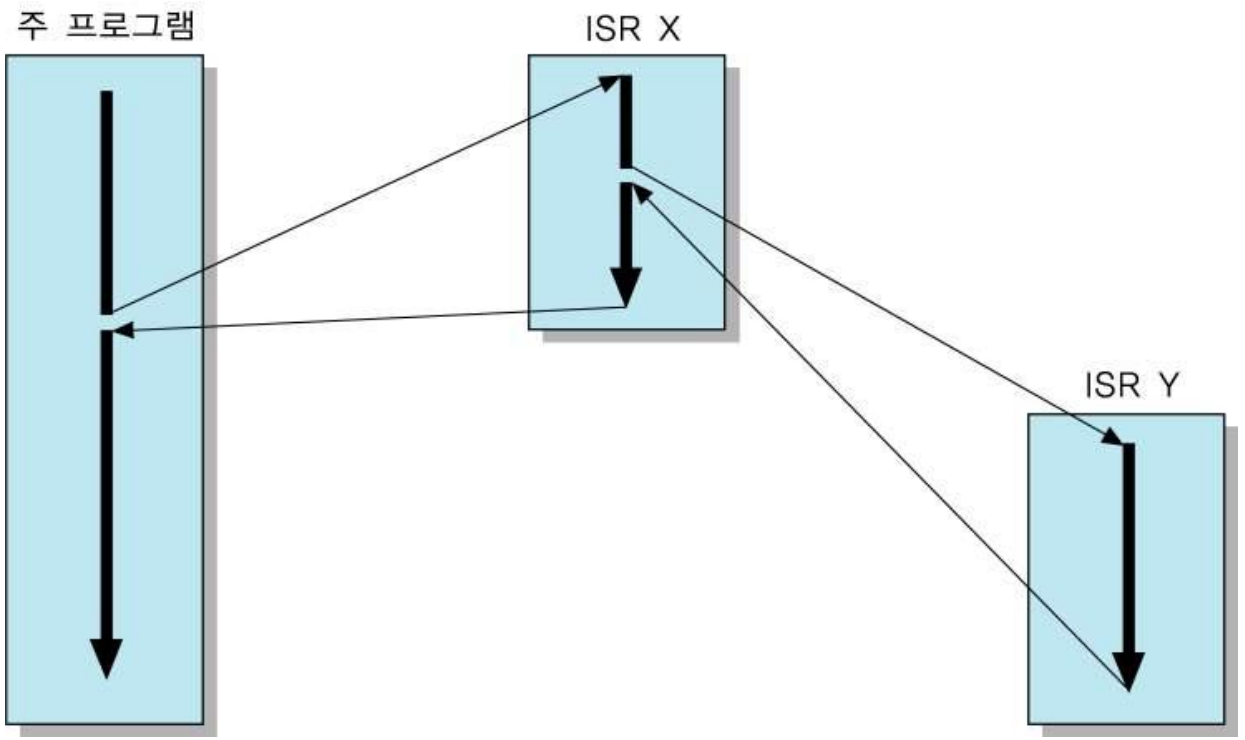
제어 플래그
- **인터럽트 (IE, Interrupt Enable flag)**
- 운영체제 (SV, supervisor mode flag)

2.2.3 인터럽트 사이클(interrupt cycle)

p75~p80

■ 다중 인터럽트 처리 방법

- ✓ 장치 *X*를 위한 ISR *X*를 처리하는 도중에
- ✓ 우선 순위가 더 높은 장치 *Y*로부터 인터럽트 요구가 들어와서 먼저 처리되는 경우에 대한 제어의 흐름



2.2.4 간접 사이클

p81~p82

■ 간접 사이클(indirect cycle)

- ✓ 명령어에 포함되어 있는 주소를 이용하여, 그 명령어 실행에 필요한 데이터의 주소를 인출하는 사이클
  - ➢ **간접 주소지정 방식**(indirect addressing mode)에서 사용
    - ❖ 예) 100번지의 내용을 CPU 내부로 가져와 A 레지스터의 값과 더한 후 B 레지스터에 저장하는 경우
- ✓ 인출 사이클과 실행 사이클 사이에 위치 → 우리가 배우는 2단계 명령어 사이클에서는 필요시에만 불림
- ✓ 간접 사이클에서 수행될 마이크로-연산 (메모리 직접 주소지정 모드의 예)
  - ① t_0: MAR ← IR(*addr*), 메모리 읽기 제어 신호
  - ② t_1: MBR ← M[MAR]
  - ③ t_2: IR(*addr*) ← MBR
- ✓ 인출된 명령어의 주소 필드 내용을 이용하여 기억장치로부터 데이터의 실제 주소를 인출하여 IR의 주소 필드에 저장 → 일부 교재에서는 t_2 단계가 없는 것으로 표현하기도 함

참고 : 4-단계 명령어 사이클 예

■ 4-단계 명령어 사이클

① **1단계: 명령어 인출 (Fetch)**

- 프로그램 카운터(PC)가 지정하는 메모리 번지에 저장되어 있는 명령어를 읽어 와서 명령어 레지스터(IR)에 저장. PC 자동 증가

② **2단계: 명령어 해독 (Decode)**

- IR 내의 명령어를 해독하여 명령어 종류에 의해 결정된 추가적인 데이터의 인출 또는 이동 등을 실행.

③ **3단계: 명령어 실행 (Execute)**

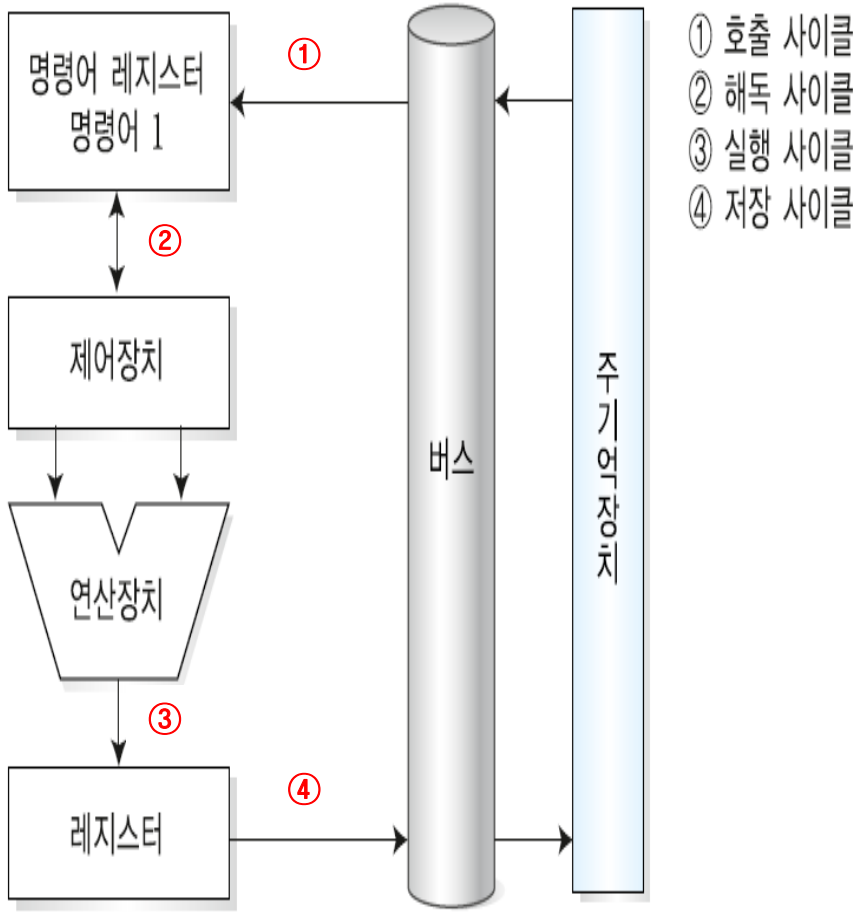
- 해독한 명령어를 실행

④ **4단계: 결과 저장 (Store)**

- ALU의 연산결과의 출력은 누산기(AC)에 저장, 필요시 시스템 버스를 경유하여 데이터 메모리에 저장
- 실행결과의 상태는 CPU 내부의 상태 레지스터(Status Register)에 기억되어 음수, 제로 등의 상태 판정에 사용

⑤ **5단계: 인터럽트 사이클**

- 인터럽트 요청 여부를 체크를 하고 없을 시에 다음 명령어 실행

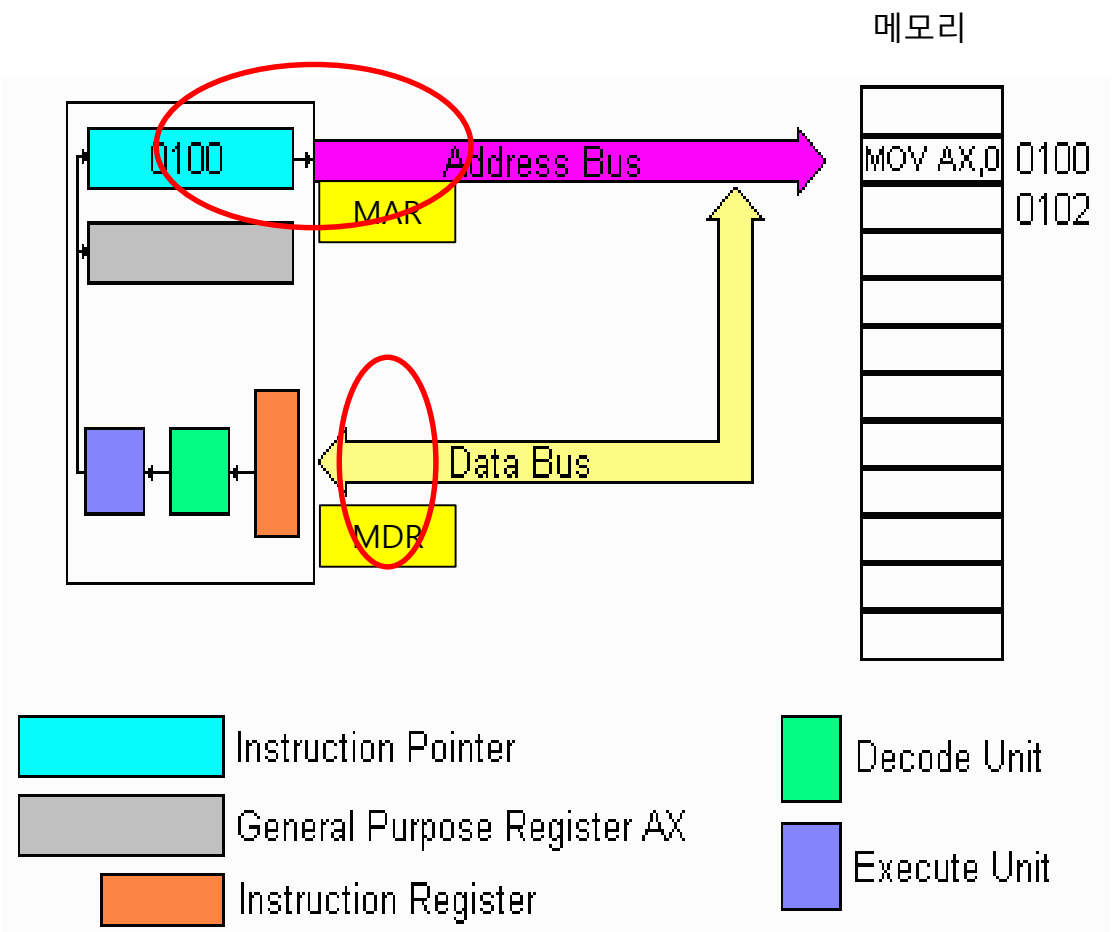


명령어의 수행 과정

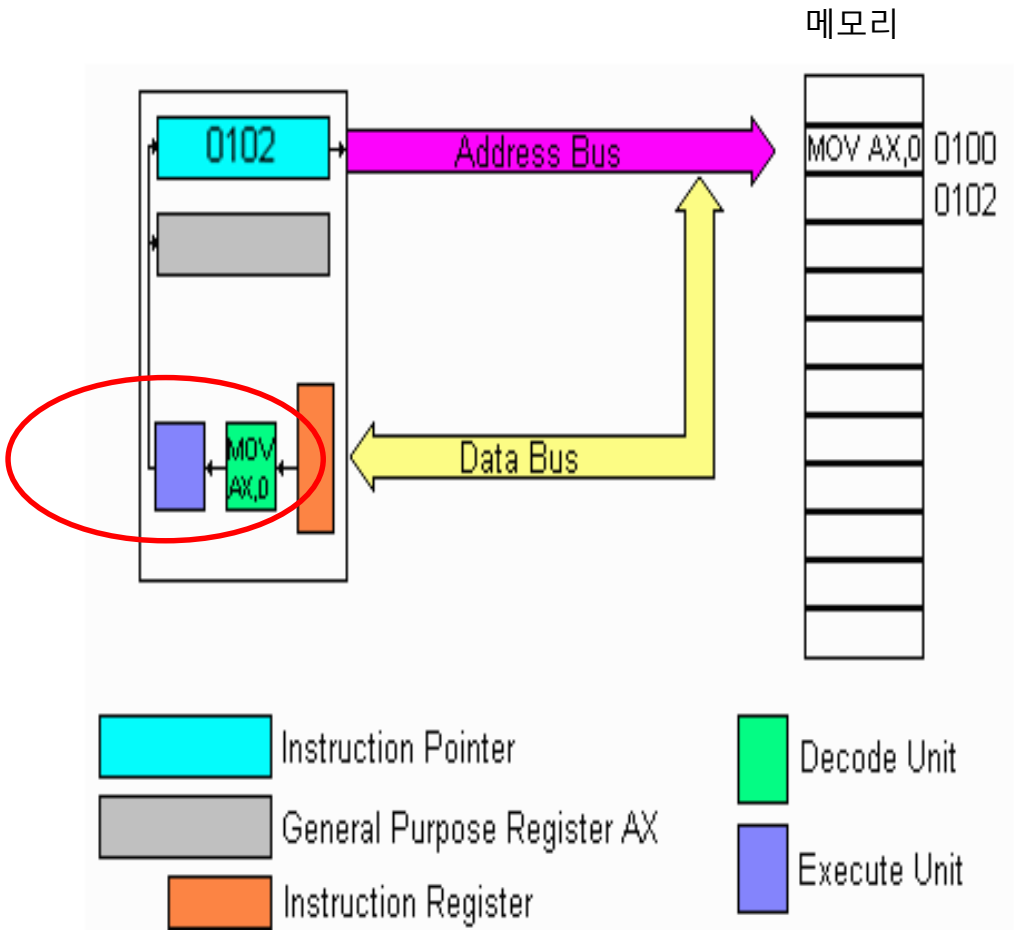
참고 1

참고 : 4-단계 명령어 사이클 예

1단계 : 명령어 인출 (Fetch)



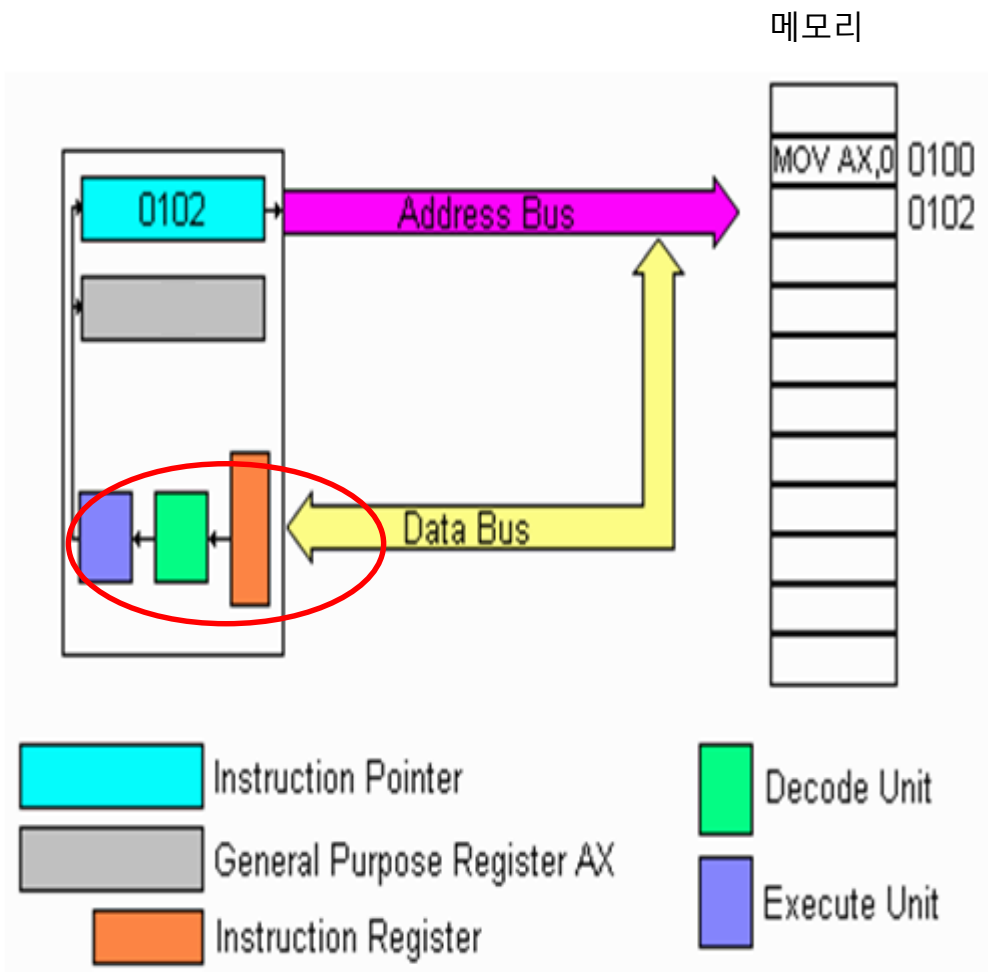
2단계 : 명령어 해독 (Decode)



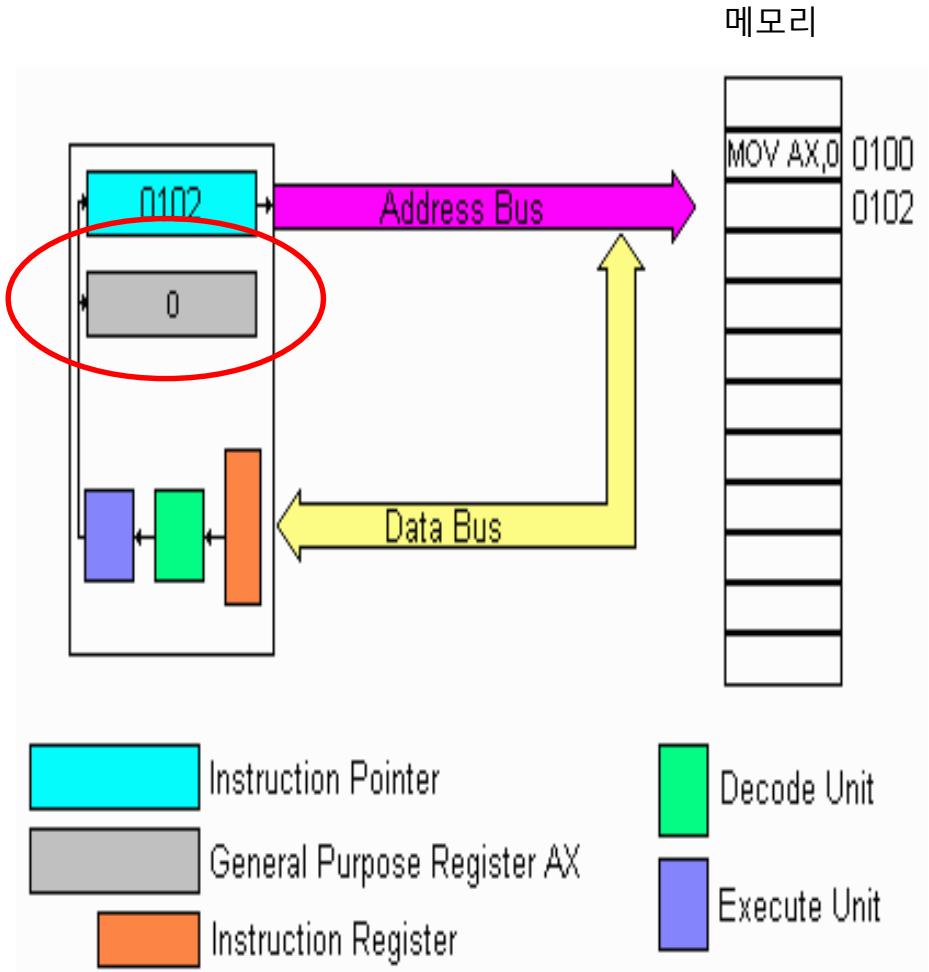
명령어 실행에 필요한 프로세서 내부 레지스터들 : PC(Instruction Pointer), AC, IR, MAR, MDR 등

참고 : 4-단계 명령어 사이클 예

3단계 : 명령어 실행(Execute)



4단계 : 결과값 저장 (Store)



Thank you